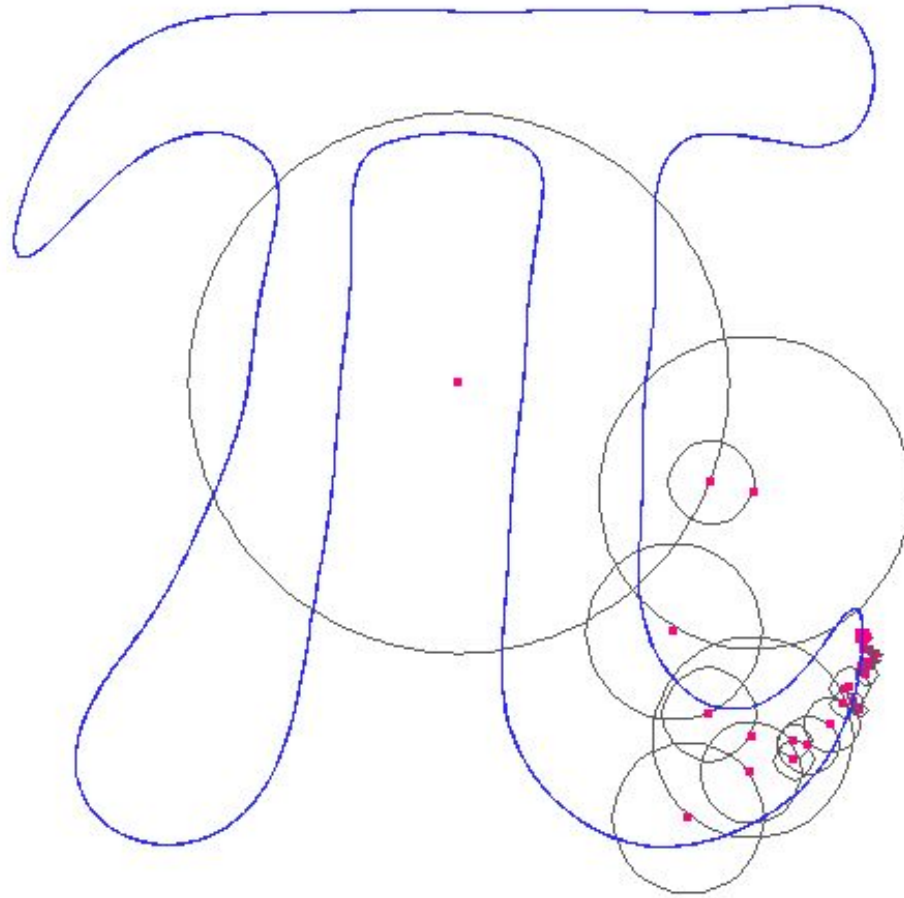




Reconstruction de courbes fermées par séries de Fourier et épicycles

TIPE 2025-2026 Cycles et boucles, Cardot Clément

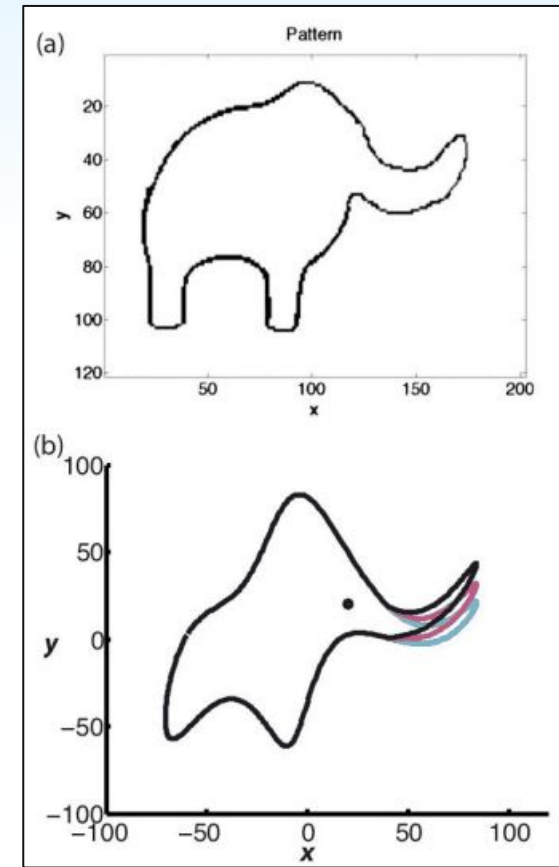
INTRODUCTION



[1]

« Avec quatre paramètres, je peux ajuster un éléphant, et avec cinq, je peux lui faire remuer la trompe. »

J. Von Neumann



[2]

Parameter	Real part	Imaginary part
$p_1=50-30i$	$B_1^x=50$	$B_1^y=-30$
$p_2=18+8i$	$B_2^x=18$	$B_2^y=8$
$p_3=12-10i$	$A_3^x=12$	$B_3^y=-10$
$p_4=-14-60i$	$A_4^x=-14$	$A_4^y=-60$
$p_5=40+20i$	Wiggle coeff.=40	$x_{cye}=y_{eye}=20$

1: [https://en.wikiquote.org/wiki/John_von_Neumann#/media/File:HD.3F.191_\(11239892036\).jpg](https://en.wikiquote.org/wiki/John_von_Neumann#/media/File:HD.3F.191_(11239892036).jpg)

2: https://publications.mpi-cbg.de/Mayer_2010_4314.pdf

Comment dessiner avec les séries de Fourier et quelles sont les limites ?

Introduction
aux séries de
Fourier

1

Lien avec le
dessin

2

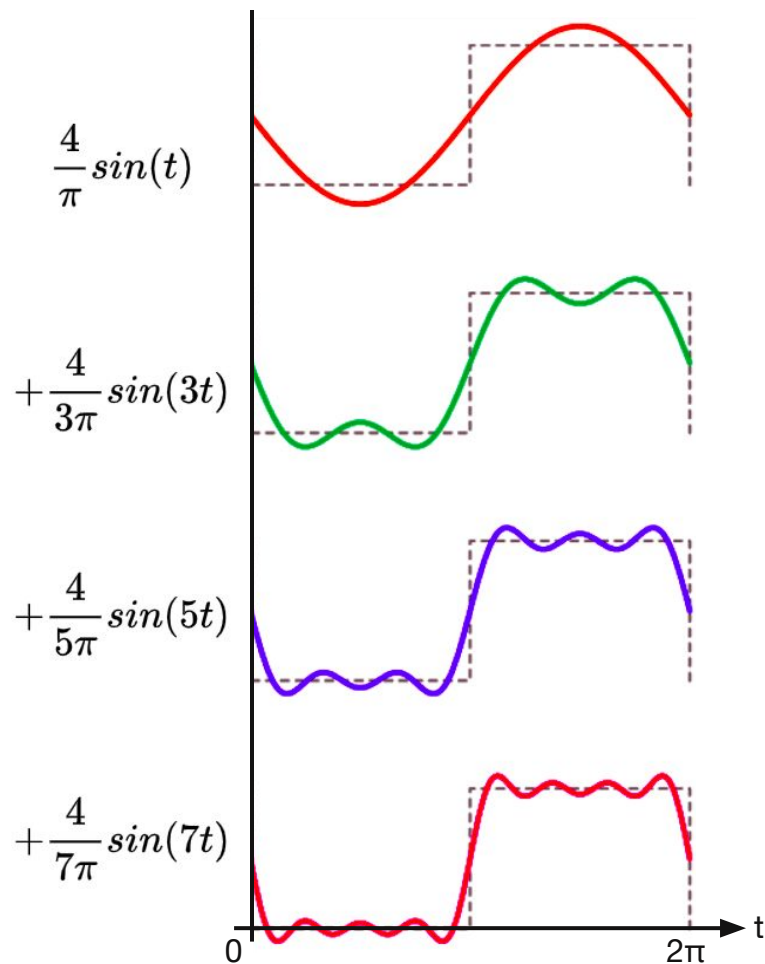
Algorithme
des voisins de
Moore

3

Limites et
évolutions

4

Thème 2025-2026 : Cycles et boucles



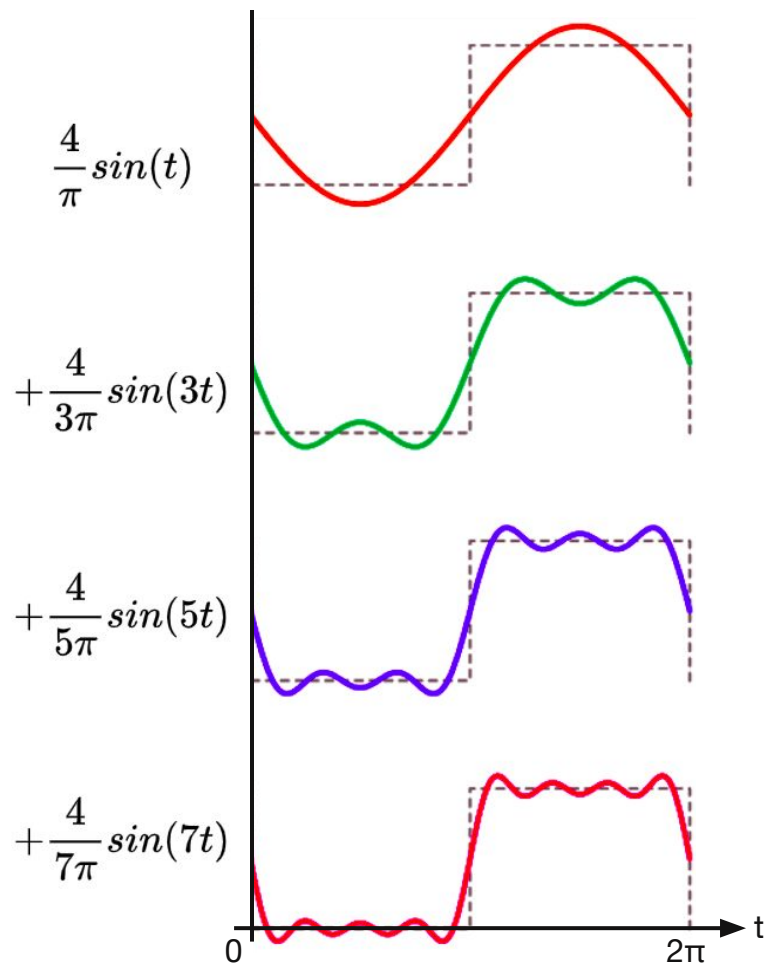
Séries de Fourier :

Soit f une fonction T -périodique, intégrable sur $[0; T]$,
 C^1 par morceaux alors pour tout t dans $[0; T]$:

$$f(t) = \sum_{n=-\infty}^{+\infty} c_n(f) e^{2i\pi n \frac{t}{T}}$$

Avec $c_n(f)$ défini par :

$$c_n(f) = \frac{1}{T} \int_0^T f(t) e^{-i2\pi n \frac{t}{T}} dt$$



Séries de Fourier :

Soit f une fonction **T-périodique**, **intégrable sur** $[0; T]$,
 C^1 par morceaux alors pour tout t dans $[0; T]$:

$$f(t) = \sum_{n=-\infty}^{+\infty} c_n(f) e^{2i\pi n \frac{t}{T}}$$

Théorème de convergence ponctuelle de Dirichlet

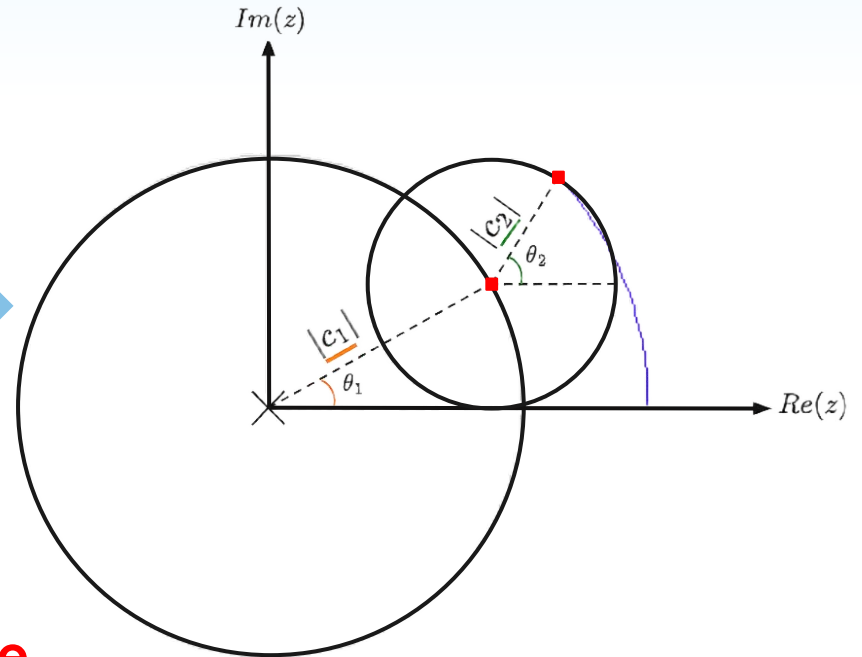
Avec $c_n(f)$ défini par :

$$c_n(f) = \frac{1}{T} \int_0^T f(t) e^{-i2\pi n \frac{t}{T}} dt$$

$$f(t) = \sum_{n \in \mathbb{Z}} c_n(f) e^{i2\pi n \frac{t}{T}}$$

$$c_n = |c_n| e^{i\theta_n}$$

$$c_n e^{i2\pi n \frac{t}{T}} = |c_n| e^{i(\theta_n + 2\pi n \frac{t}{T})}$$

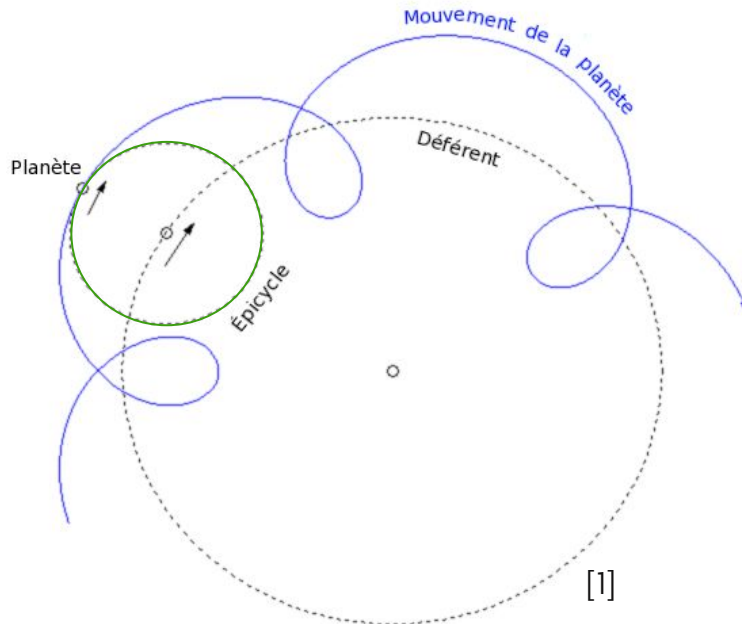


Un coefficient = Un cercle



```

302 typedef struct {
303     double cx, cy;    // centre (double maintenant)
304     double rayon;    // rayon (Module du coeff de Fourier)
305     double angle;    // phase initiale (argument du coeff de Fourier)
306     double vitesse;  // vitesse angulaire (2*PI*n pour le n-ieme coeff, en rad/s)
307     int sens;        // sens de rotation : +1 ou -1 (dépend du signe de n)
308 } Cercle;
    
```



The Mathematical Power of Epicyclical Astronomy

By Norwood Russell Hanson*

THIS PAPER has two objectives. The first is to describe more graphically than has yet been done the elegance of the ancient technique of epicycle-on-deferent. The second is to expose several historians — namely, that the somehow connected with its formally compared with the powerful calculi of epicycle-on-deferent astronomy appear attempt to predict planetary perturbations. Actually, these objectives will be achieved at all. To see the comprehensiveness of the geometrical device just is to see its elegance.

First then, a graphic account of the Let Ptolemy himself introduce the ... the epicycle is made to move in a circle concentric with the Zodiac ... moves on the epicycle at a velocity ... on that part of the epicycle farthest from the Earth. Ptolemy here describes that example of the staple illustration in history of science. It is said of Herakleides that he envied Venus. Thus, Chalcidius:

* Indiana University. I am indebted to Dr. F. Smithies, of St. John's College, Cambridge University, for help with the formal parts of this paper. So prevalent is this view that as late as 1880 De Morgan had to write: "On this theory of epicycles ... the common notion is that it was a cumbersome and useless apparatus, thrown away by the moderns" (*Dictionary of Greek and Roman Biography*, compiled by Sir William Smith [London, 1880], s.v. Claudius Ptolemaeus, p. 576). Dampier does not help to correct "the common notion" forcibly or clearly enough. He writes: "Its [the theory of Hipparchus and Ptolemy] one fault from

15

EPICYCLICAL ASTRONOMY

153

hypothesis could be made equivalent to the elliptical orbit hypothesis, independently discovered useful for calculating Mars' longitudes.¹⁵ By altering the speed of the planet's east-to-west motion on an epicycle travelling west-to-east, one can approximate to a triangular figure (Figure 6).

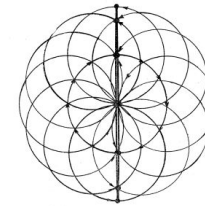


FIGURE 5

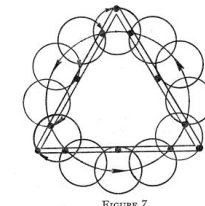


FIGURE 7

Indeed, one can take this to its rectilinear limit. When confronted with this triangular orbit follows (Figure 8), several historians are

¹⁵ *Ibid.*, ch. 59. Kepler does not actually undertake the detailed construction we have just set out above; but inasmuch as the librations in question are always "librations in longitude"



EPICYCLICAL ASTRONOMY

157

E. Note that any form of retrograde motion corresponds simply to taking T (or k) negative.

F. A motion given by the superposition of n epicycles would be given by an equation of this form:

$$z = a_1 e^{i\omega_1 t} + a_2 e^{i\omega_2 t} + \dots + a_n e^{i\omega_n t}$$

G. Now suppose that we are just given a periodic motion in the plane by the equation:

$$z = f(t)$$

We may assume now that the period of motion is 2π (even if it becomes necessary to change the time scale somewhat).

Suppose that $f(t)$ is a sufficiently well-behaved function [it will be enough that $f(t)$ is continuous and of bounded variation—this is a natural enough condition to impose; that $f(t)$ is taken as continuous and of bounded variation means only that the resultant orbit is a continuous curve of finite length.] Then it is well known that we can write:

$$f(t) = c_0 + (c_1 e^{it} + c_{-1} e^{-it}) + (c_2 e^{2it} + c_{-2} e^{-2it}) + \dots = \sum_{n=-\infty}^{\infty} c_n e^{int}$$

this series being uniformly convergent.

Let us now write

$$S_N(t) = \sum_{n=-N}^N c_n e^{int}$$

Choose now some small number $\epsilon > 0$. Since the series is uniformly convergent, we can now find N_ϵ such that

$$|f(t) - S_N(t)| < \epsilon$$

for all $N \geq N_\epsilon$ and all t . In other words, if we consider the two orbits:

$$z = f(t), \quad z = S_N(t),$$

the distance between corresponding points of these two orbits remains less than ϵ for all time.

Thus the original orbit, $z = f(t)$, can be replaced, with as small a loss in accuracy as we please, by an orbit $z = S_N(t)$; that is, some finite superposition of epicycles.

H. The function defining a square or a triangular "orbit" satisfies these conditions completely; the above considerations therefore apply without qualification to these special cases illustrated earlier in this paper.

I. The superpositions we have used so far are special ones; for the periods of the epicycles are:

$$\pm 2\pi, \pm \pi, \pm 2/3\pi, \pm 1/2\pi, \pm 2/5\pi, \dots$$

In particular, they are commensurable with one another.

J. However, it must be stressed here that even non-periodic orbits can be represented by such a superposition of epicycles, provided only that we allow incommensurable periods. The basic theorem would be:

Let $z = f(t)$ be a complex-valued function of the real variable t , and suppose

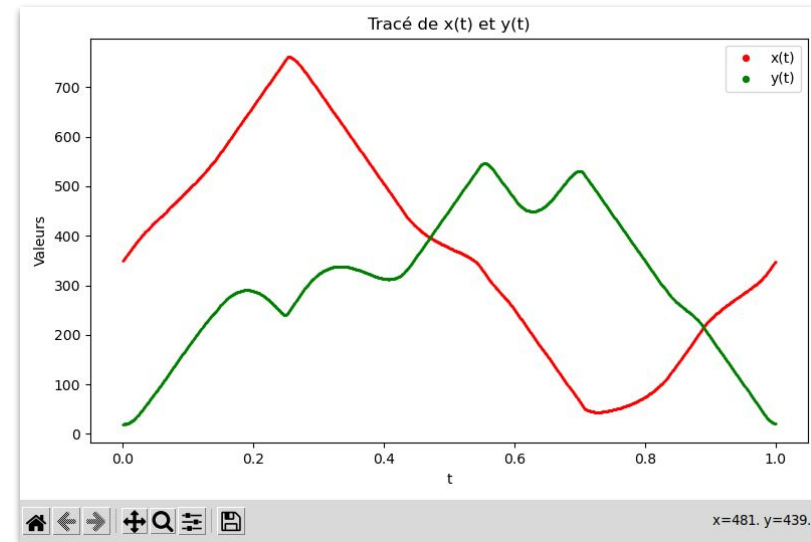
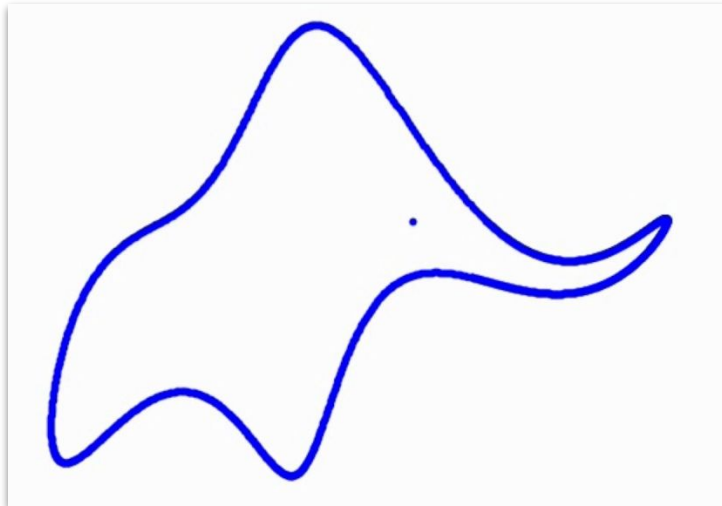
$$S_N(t) = \sum_{n=-N}^N c_n e^{int}$$

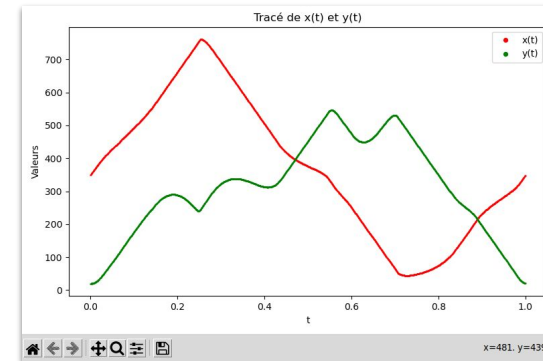
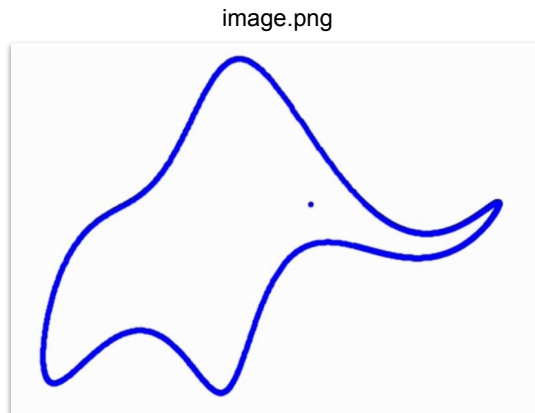
1: https://fr.wikipedia.org/wiki/%C3%89picycle#/media/Fichier:Epicycle_et_deferent.png

2: https://web.math.princeton.edu/~epwyw/F22FRS/hanson_epicycles.pdf

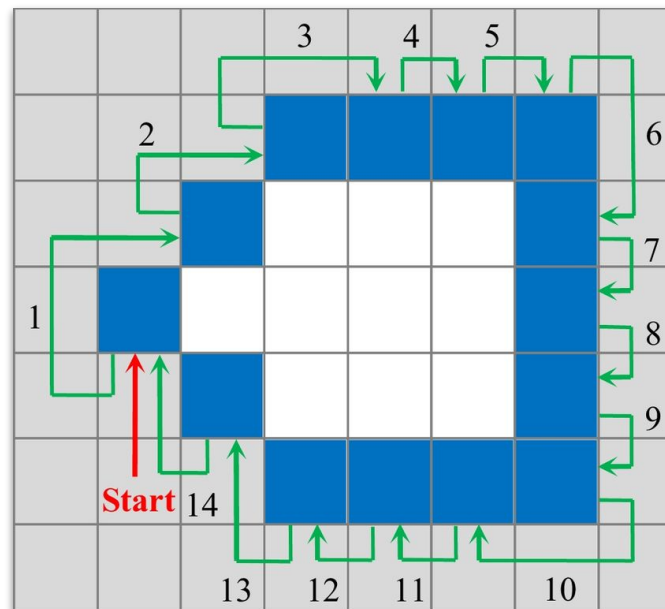
$$c_n(f) = \frac{1}{T} \int_0^T f(t) e^{-i2\pi n \frac{t}{T}} dt$$

image.png



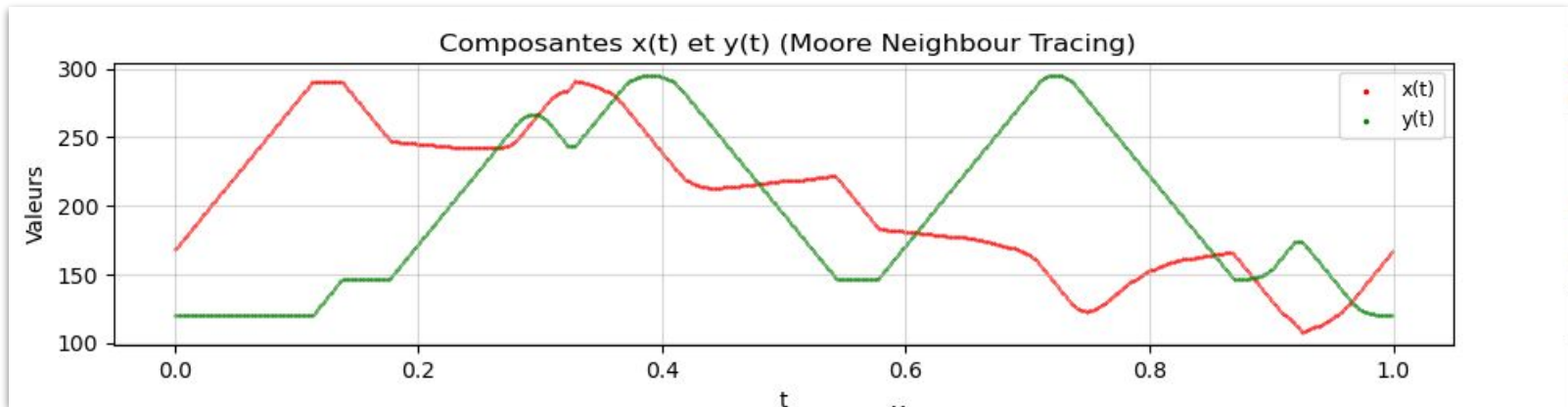


Algorithme des voisins de Moore

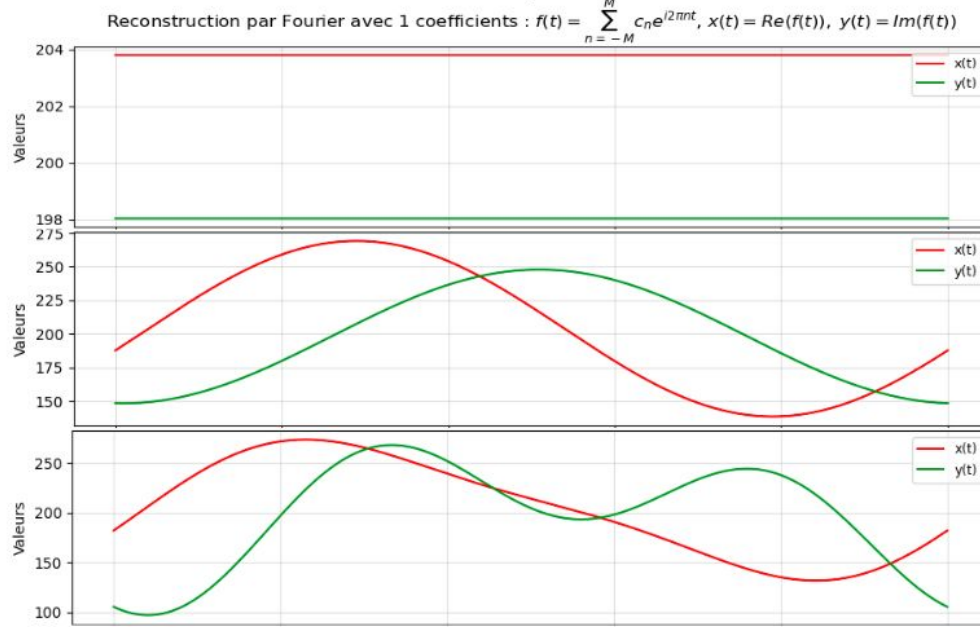
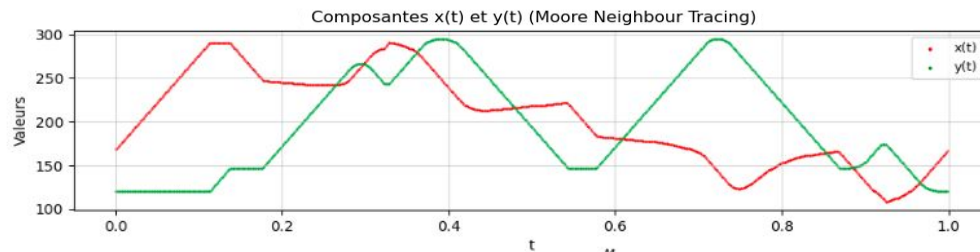


https://www.researchgate.net/figure/Principle-of-Moores-neighbor-contour-tracing-algorithm_fig5_374555715

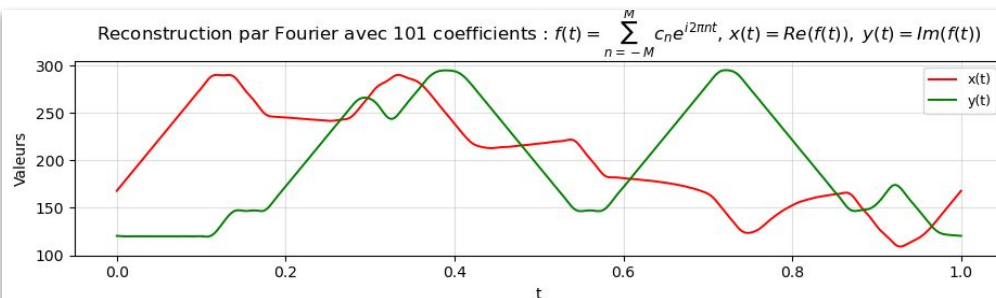
Forme de π :



$$c_n(f) = \frac{1}{T} \int_0^T f(t) e^{-i2\pi n \frac{t}{T}} dt$$



⋮

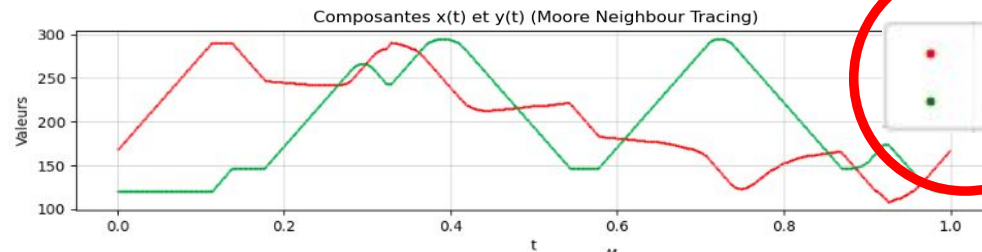


1 coefficient

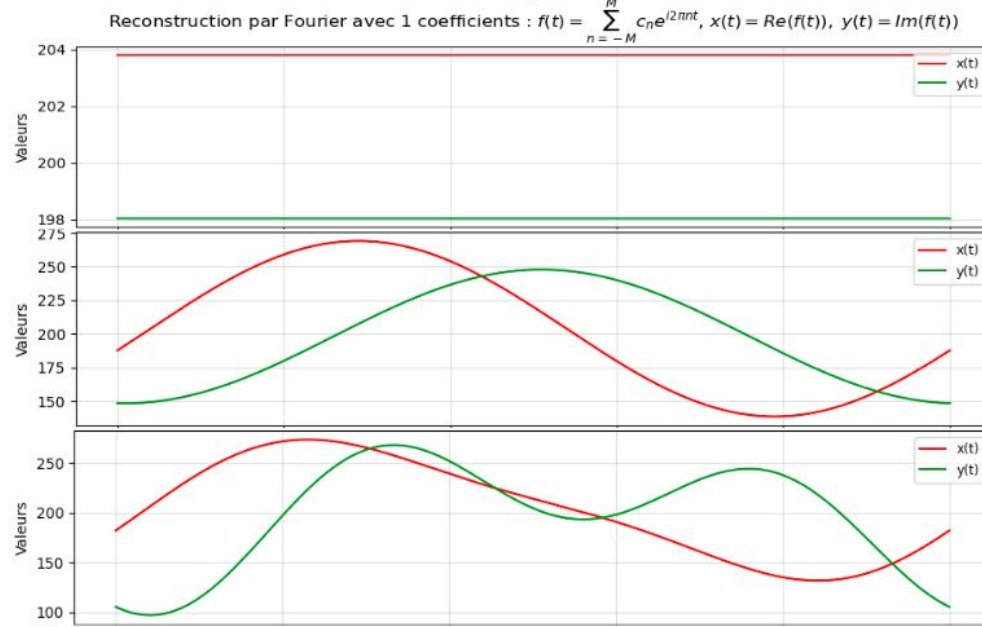
3 coefficients

5 coefficients

101 coefficients



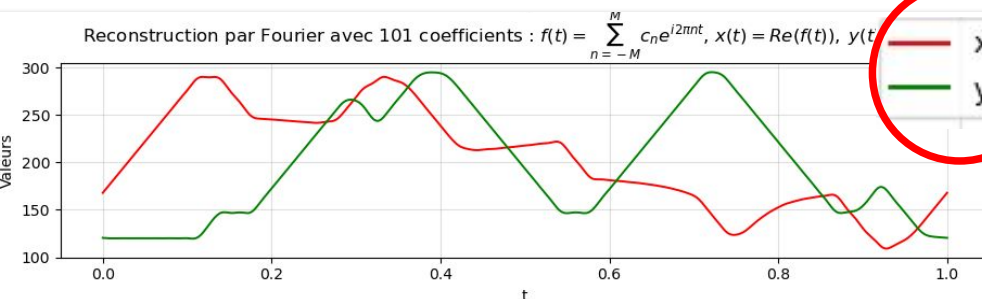
Points



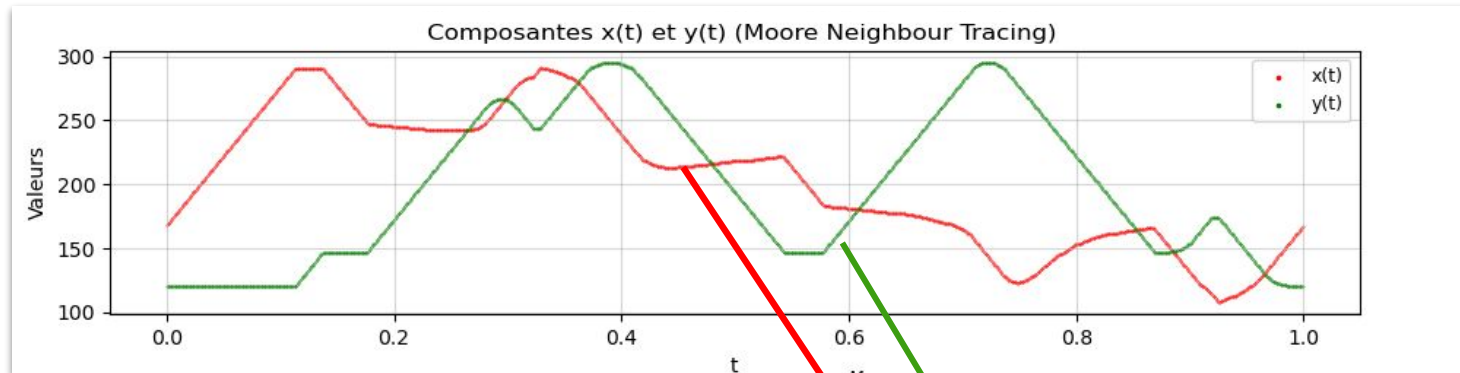
1 coefficient

3 coefficients

5 coefficients



**Courbe
continue
101 coefficients**



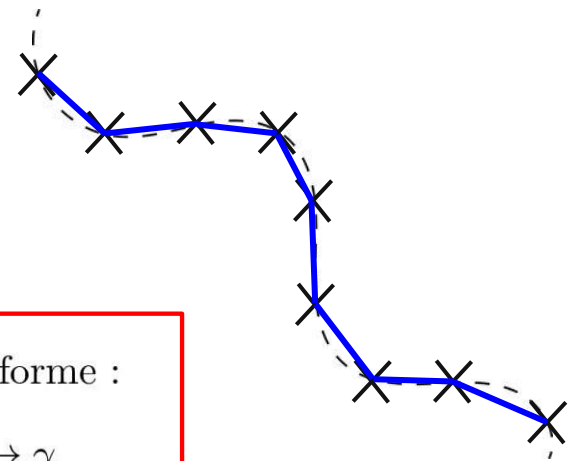
$$P_k = (x_k, y_k) \in \mathbb{R}^2, \quad k \in [0, \dots, N-1] \quad t_k := \frac{k}{N}, \quad \gamma(t_k) = \boxed{x_k} + i\boxed{y_k}$$

γ est une fonction affine sur chaque sous-intervalle $[t_k, t_{k+1}]$.

$$\text{On considère la somme partielle } S_M(\gamma)(t) = \sum_{n=-M}^M c_n(\gamma) e^{i2\pi n t}.$$

Par le théorème de Dirichlet, on obtient la convergence uniforme :

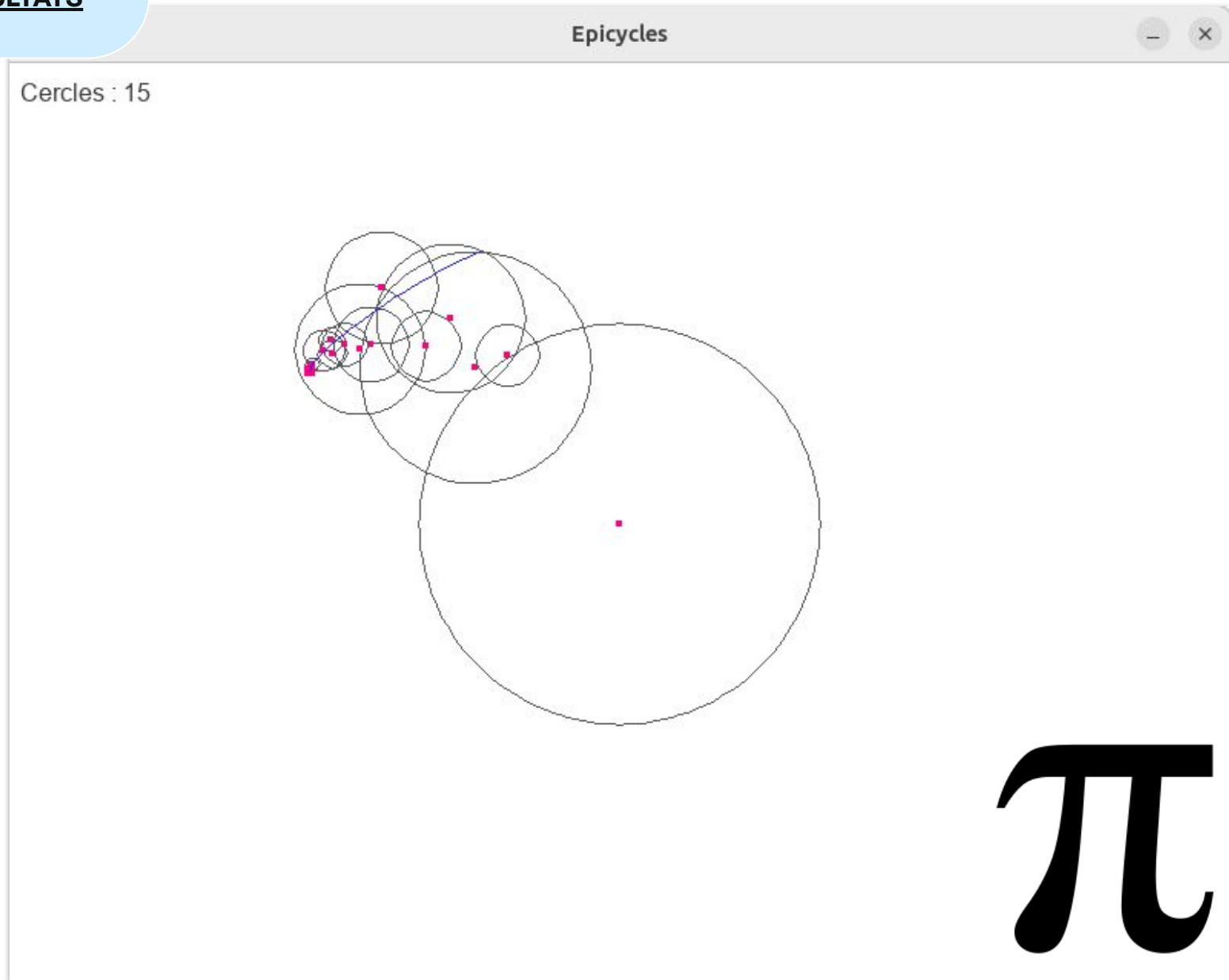
$$\gamma \in C^0([0, 1]) \cap C^1_{\text{morceaux}}([0, 1]), \text{ on a donc } S_M(\gamma)(t) \xrightarrow[M \rightarrow \infty]{CVU} \gamma$$



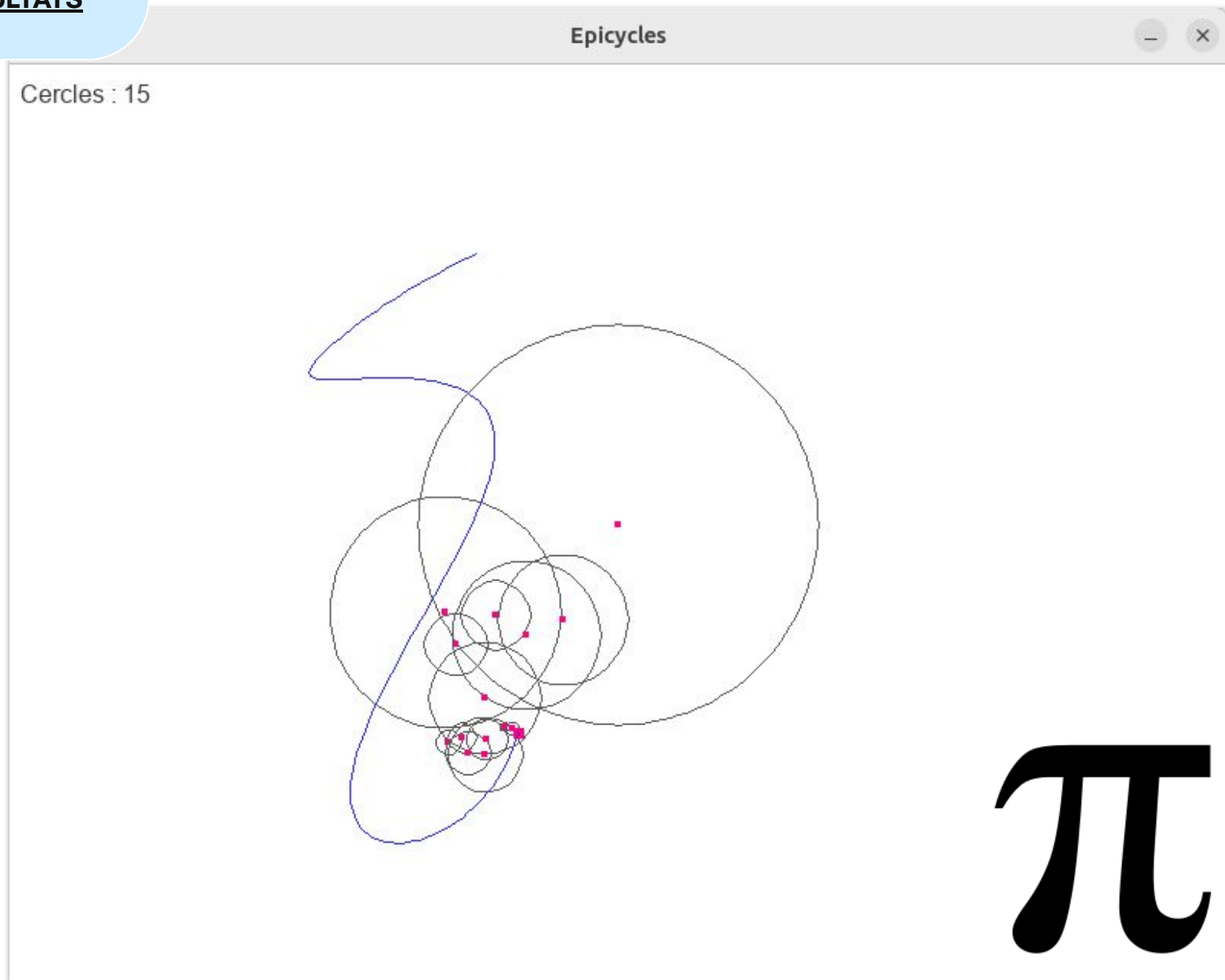
(15) Coeffs de Fourier

```
1 n, re, im|
2 -7, 1.221139, -1.931306
3 -6, 6.145095, -1.996594
4 -5, -4.100920, -1.819058
5 -4, -0.013597, 11.179418
6 -3, -14.410933, 15.960539
7 -2, 12.281872, -14.625393
8 -1, -6.598456, 6.809617
9 0, 203.795937, 198.042475
10 1, -9.519832, -56.493792
11 2, -17.956035, -28.297651
12 3, 9.471992, 4.216437
13 4, -15.507438, -4.931939
14 5, -3.638235, -1.881227
15 6, 4.169340, -5.191361
16 7, -0.689830, 1.797221
```

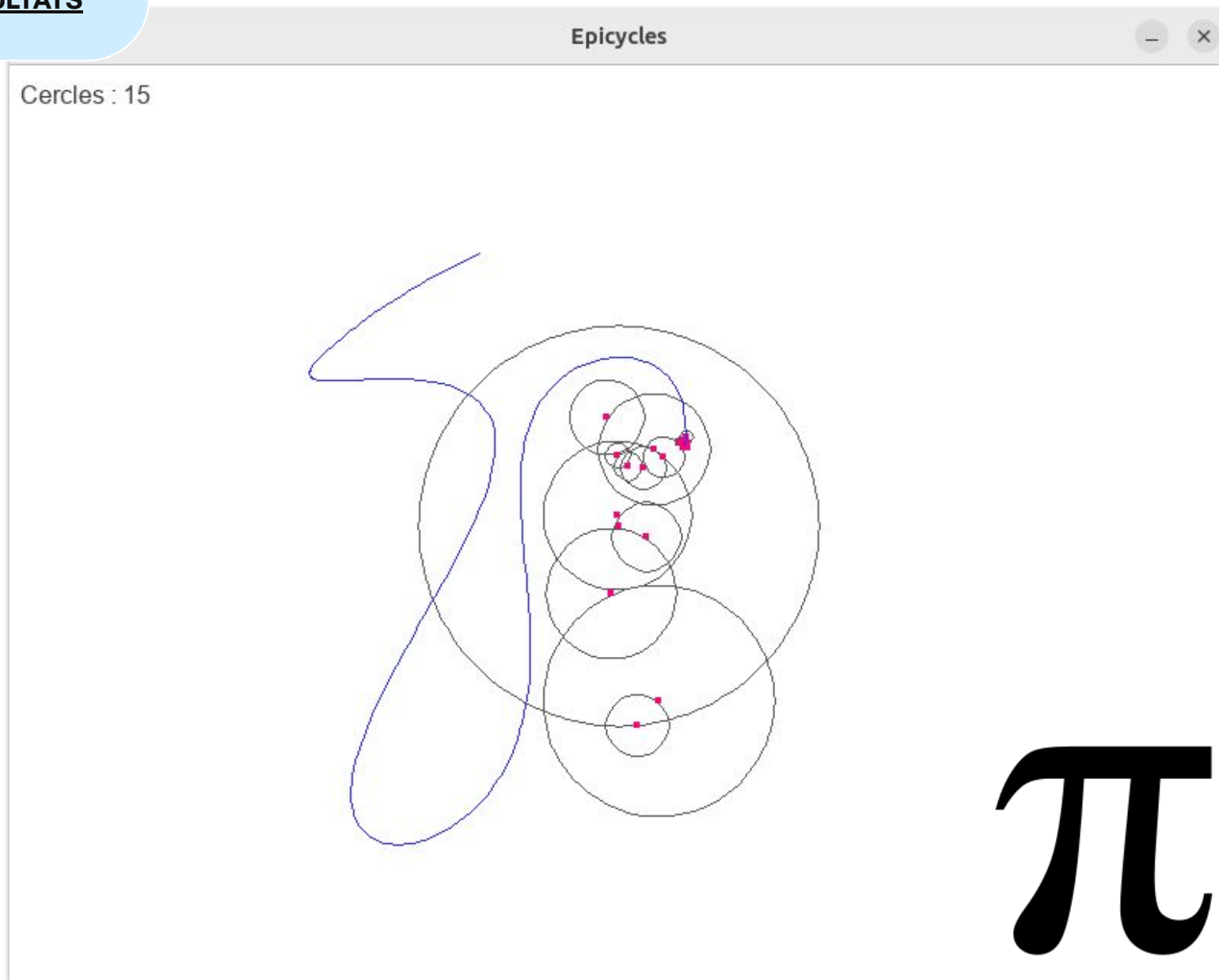
RESULTATS



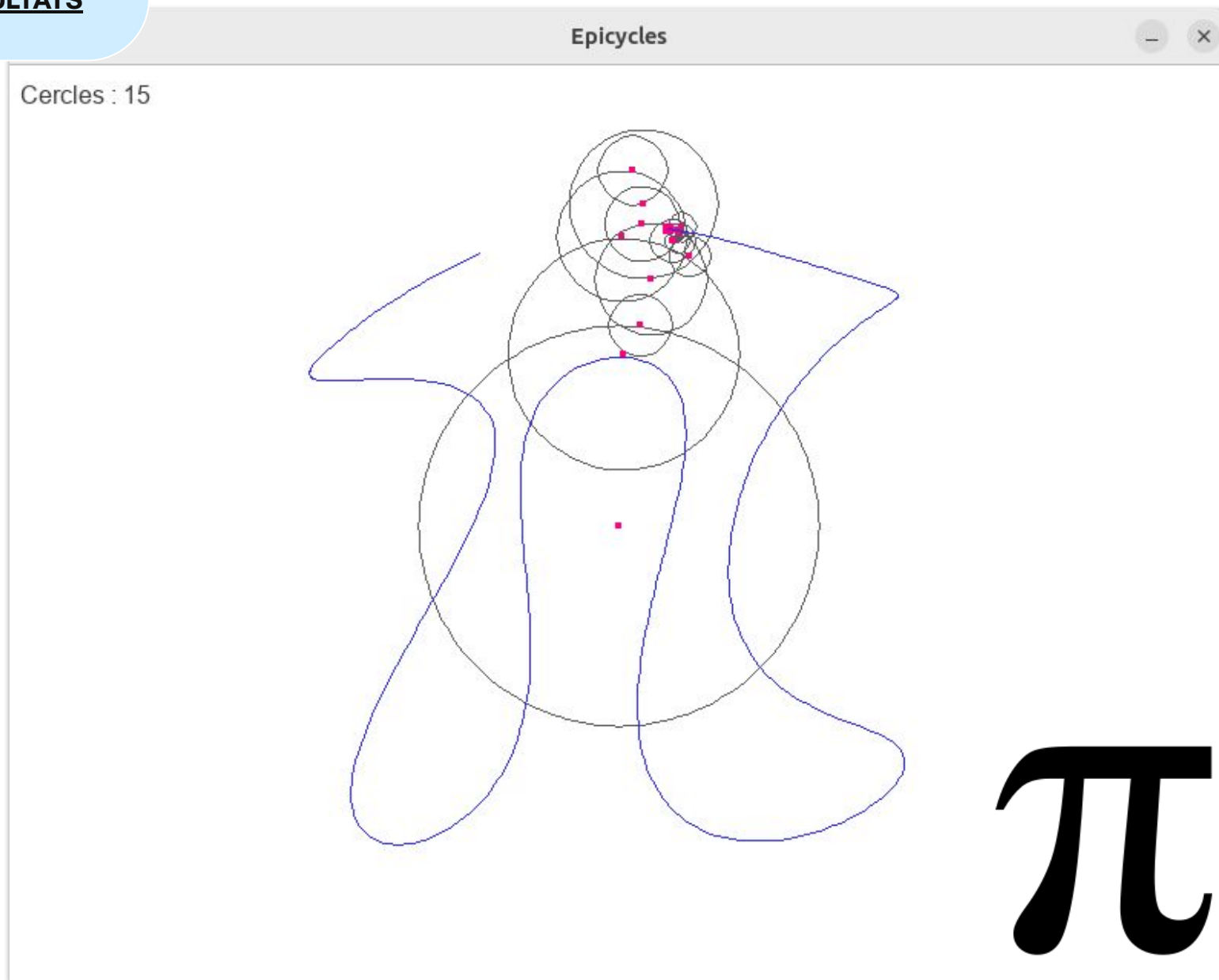
RESULTATS



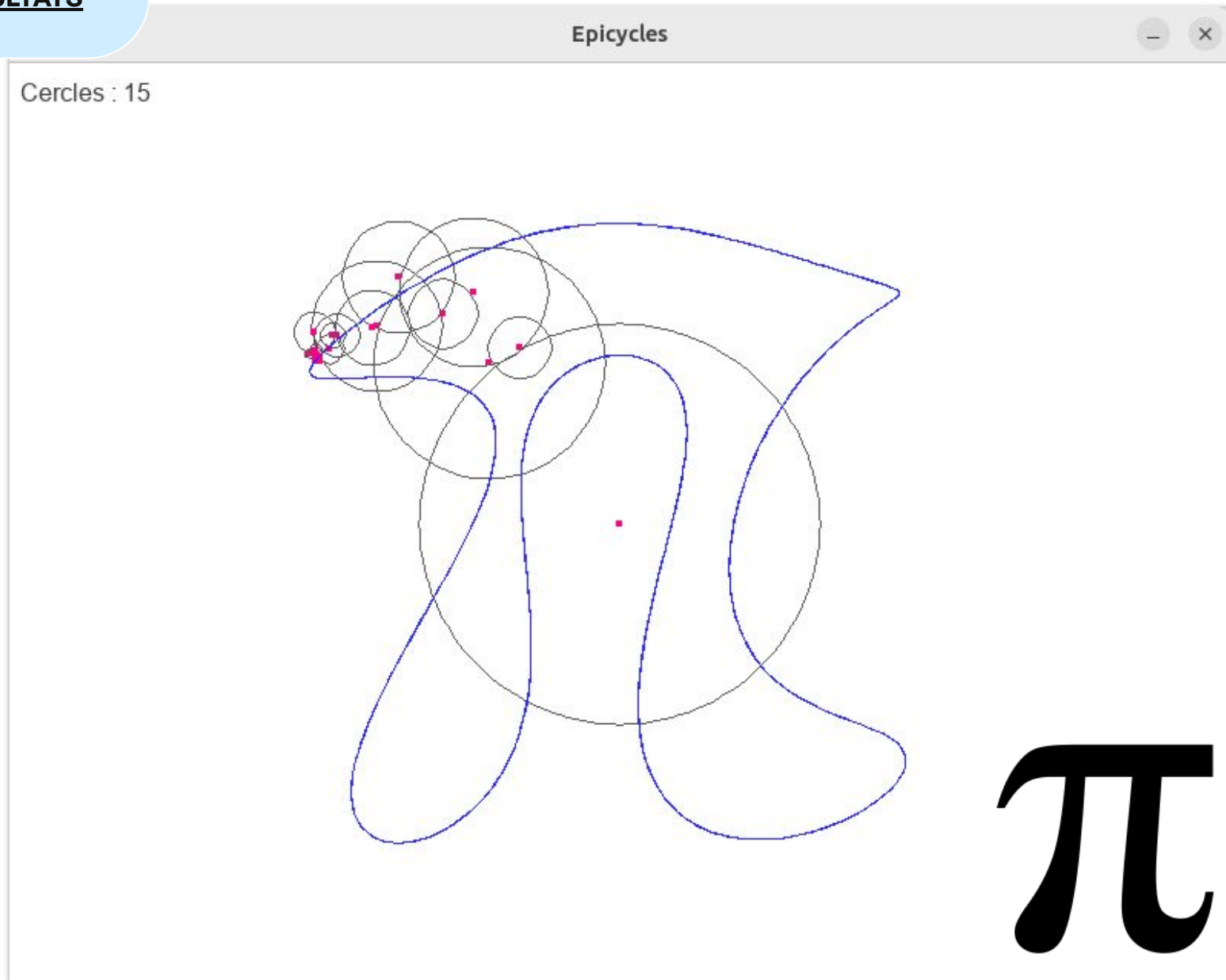
RESULTATS



RESULTATS



RESULTATS

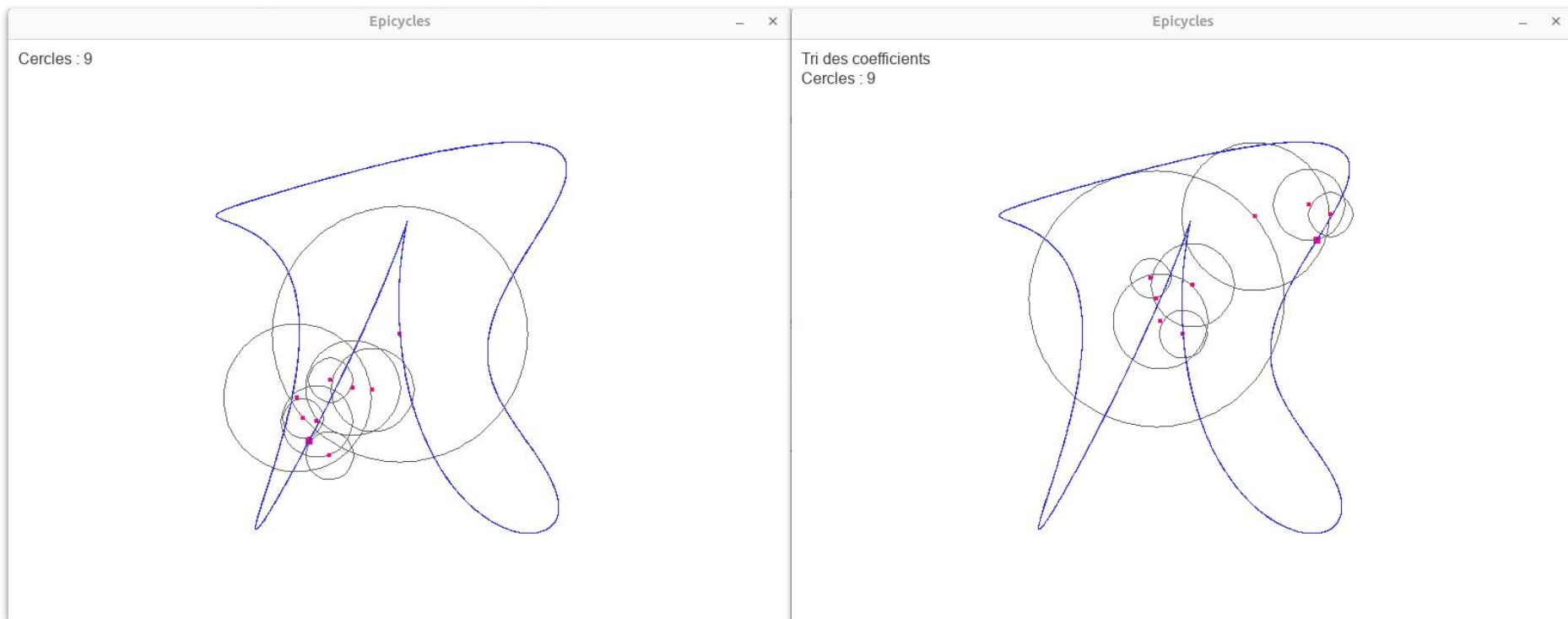


Sélection des coefficients de Fourier :

$|c_n|$ = importance géométrique, grand $|c_n| \rightarrow$ grande contribution

$$f_M(t) = \sum_{n=-M}^M c_n e^{i2\pi n t}$$

Seulement les $2M+1$ plus grands coefficients sur les 10M calculés



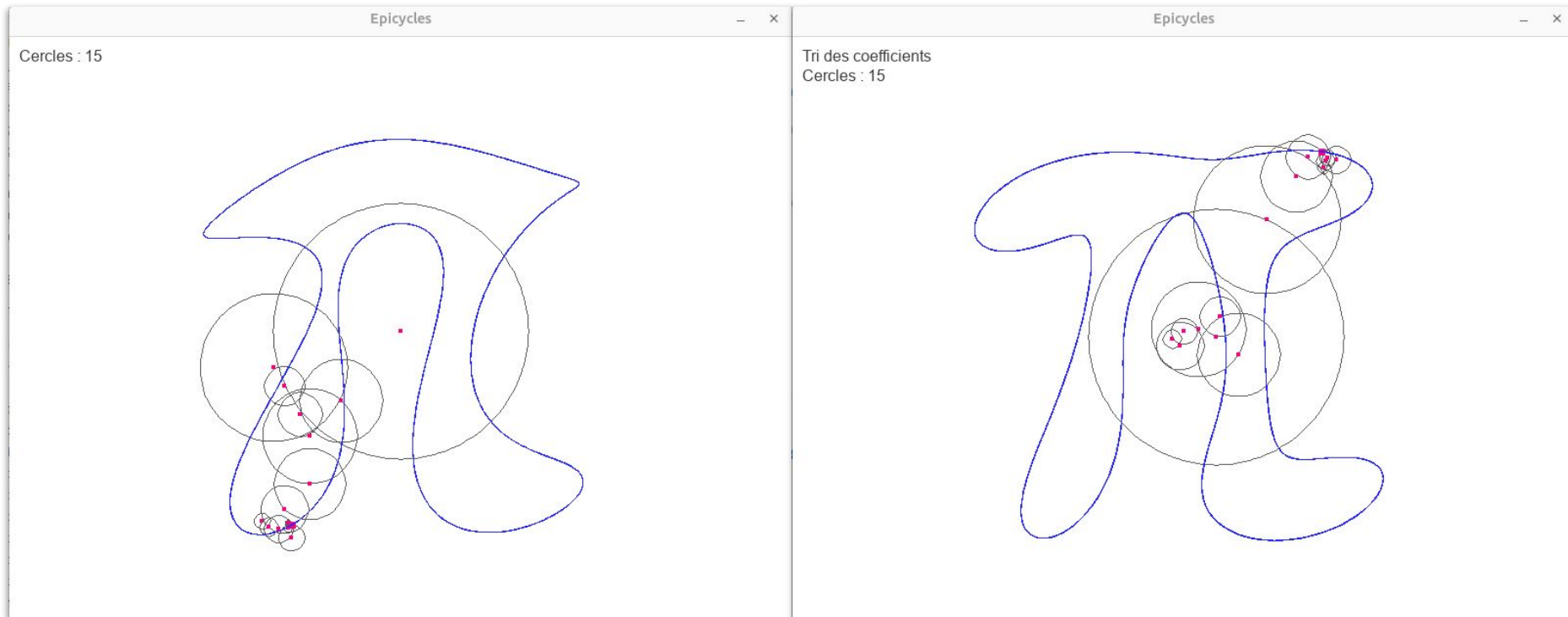
π

Sélection des coefficients de Fourier :

$|c_n|$ = importance géométrique, grand $|c_n| \rightarrow$ grande contribution

$$f_M(t) = \sum_{n=-M}^M c_n e^{i2\pi n t}$$

Seulement les $2M+1$ plus grands coefficients sur les
10M calculés



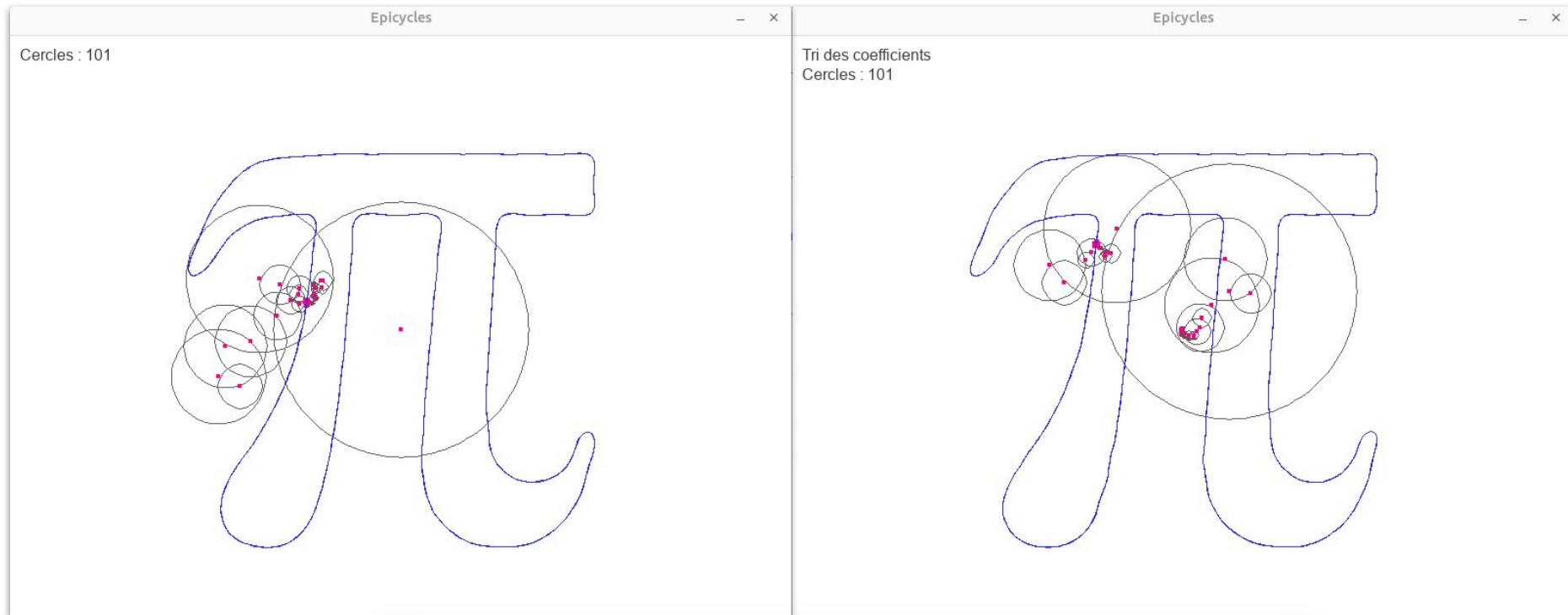
π

Sélection des coefficients de Fourier :

$|c_n|$ = importance géométrique, grand $|c_n| \rightarrow$ grande contribution

$$f_M(t) = \sum_{n=-M}^M c_n e^{i2\pi n t}$$

Seulement les $2M+1$ plus grands coefficients sur les
10M calculés



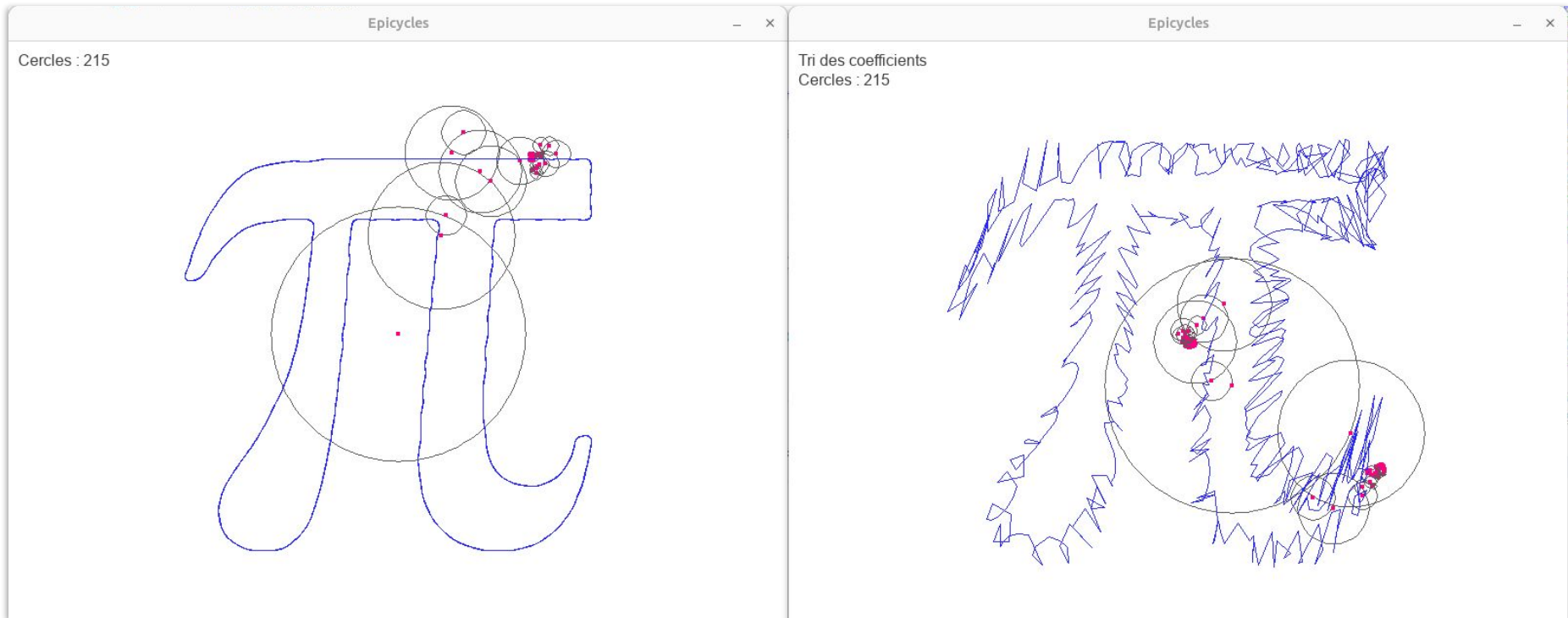
π

Sélection des coefficients de Fourier :

$|c_n|$ = importance géométrique, grand $|c_n| \rightarrow$ grande contribution

$$f_M(t) = \sum_{n=-M}^M c_n e^{i2\pi n t}$$

Seulement les $2M+1$ plus grands coefficients sur les
10M calculés



π

```
crdtclément@crdtclément-Thin-GF63-12UCX:~/Documents/MPI KHUBE/TIPE/PAUSE ET  
NBR CERCLES$ ./Dessin_final 0 TEST_2.png 2
```

TIPE 2025-2026, MPI* Lycée Louis Thuillier:Cardot Clément

- Étape 1 : Extraction du contour de l'image TEST_2.png
Nombre de points du contour : 1180
Fichier eq_parametrique.csv écrit.
- Étape 2 : Calcul des coefficients de Fourier (M=2)
Fichier coeffs_fourier.csv écrit.
- Étape 3 : Tracé de la forme avec des epicycles
SCALE = 1.012048
Affichage en cours...

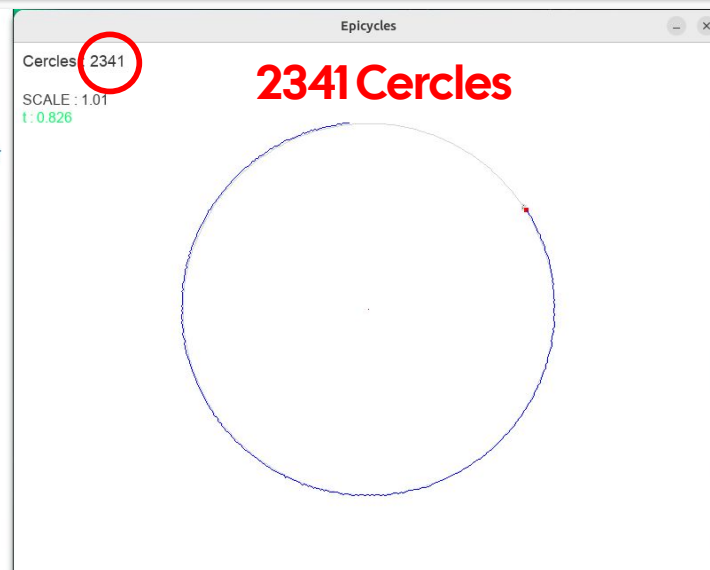
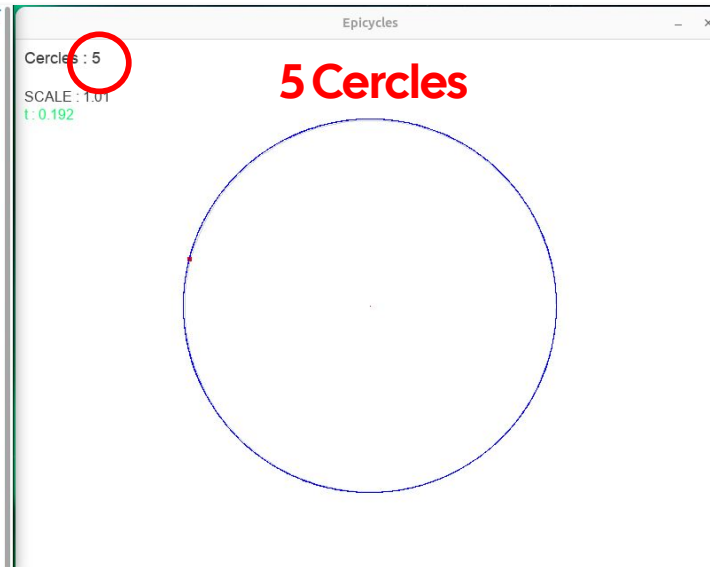
Affichage en cours...

Fin affichage.
Fermeture du programme.

```
crdtclément@crdtclément-Thin-GF63-12UCX:~/Documents/MPI KHUBE/TIPE/PAUSE ET  
NBR CERCLES$ ./Dessin_final 0 TEST_2.png 1170
```

TIPE 2025-2026, MPI* Lycée Louis Thuillier:Cardot Clément

- Étape 1 : Extraction du contour de l'image TEST_2.png
Nombre de points du contour : 1180
Fichier eq_parametrique.csv écrit.
- Étape 2 : Calcul des coefficients de Fourier (M=1170)
ATTENTION : M trop grand !(> 3/4 du nombre de points, pas fiable)
Fichier coeffs_fourier.csv écrit.
- Étape 3 : Tracé de la forme avec des epicycles
SCALE = 1.012048
Affichage en cours...



2361 Cercles

Affichage en cours...

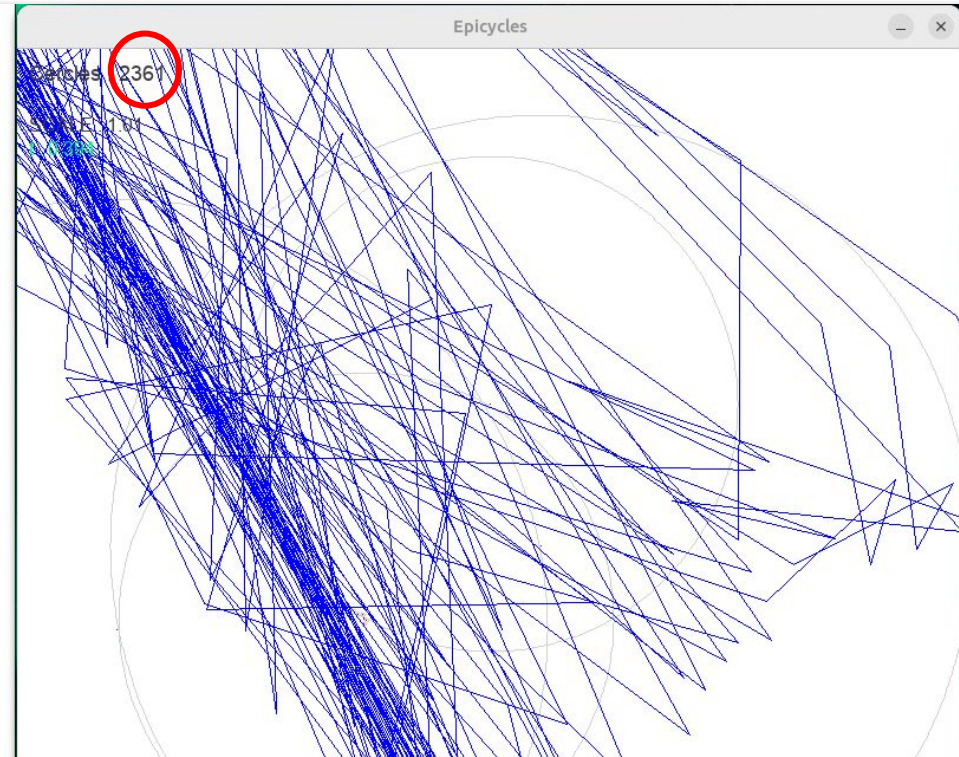
Fin affichage.

Fermeture du programme.

```
crdtclément@crdtclément-Thin-GF63-12UCX:~/Documents/MPI KHUBE/TIPE/PAUSE ET  
NBR CERCLES$ ./Dessin_final 0 TEST_2.png 1180
```

TIPE 2025-2026, MPI* Lycée Louis Thuillier:Cardot Clément

- Étape 1 : Extraction du contour de l'image TEST_2.png
Nombre de points du contour 1180
Fichier eq_parametrique.csv écrit.
- Étape 2 : Calcul des coefficients de Fourier (M=1180)
ATTENTION : M trop grand !(> 3/4 du nombre de points, pas fiable)
Fichier coeffs_fourier.csv écrit.
- Étape 3 : Tracé de la forme avec des epicycles
SCALE = 1.012048
Affichage en cours...



Problème d'échantillonnage :

$$\begin{aligned}
 c_n &= \frac{1}{N} \sum_{k=0}^{N-1} f_k e^{-2i\pi n \frac{k}{N}} \\
 &= \frac{1}{N} \sum_{k=0}^{N-1} f_k e^{-2i\pi n \frac{k}{N}} \underbrace{e^{2i\pi N \frac{k}{N}}}_{=1} \\
 &= \frac{1}{N} \sum_{k=0}^{N-1} f_k e^{-2i\pi(n+N) \frac{k}{N}} \\
 &= c_{n+N} \quad \text{Periodicité}
 \end{aligned}$$



coeffs_fourier.csv

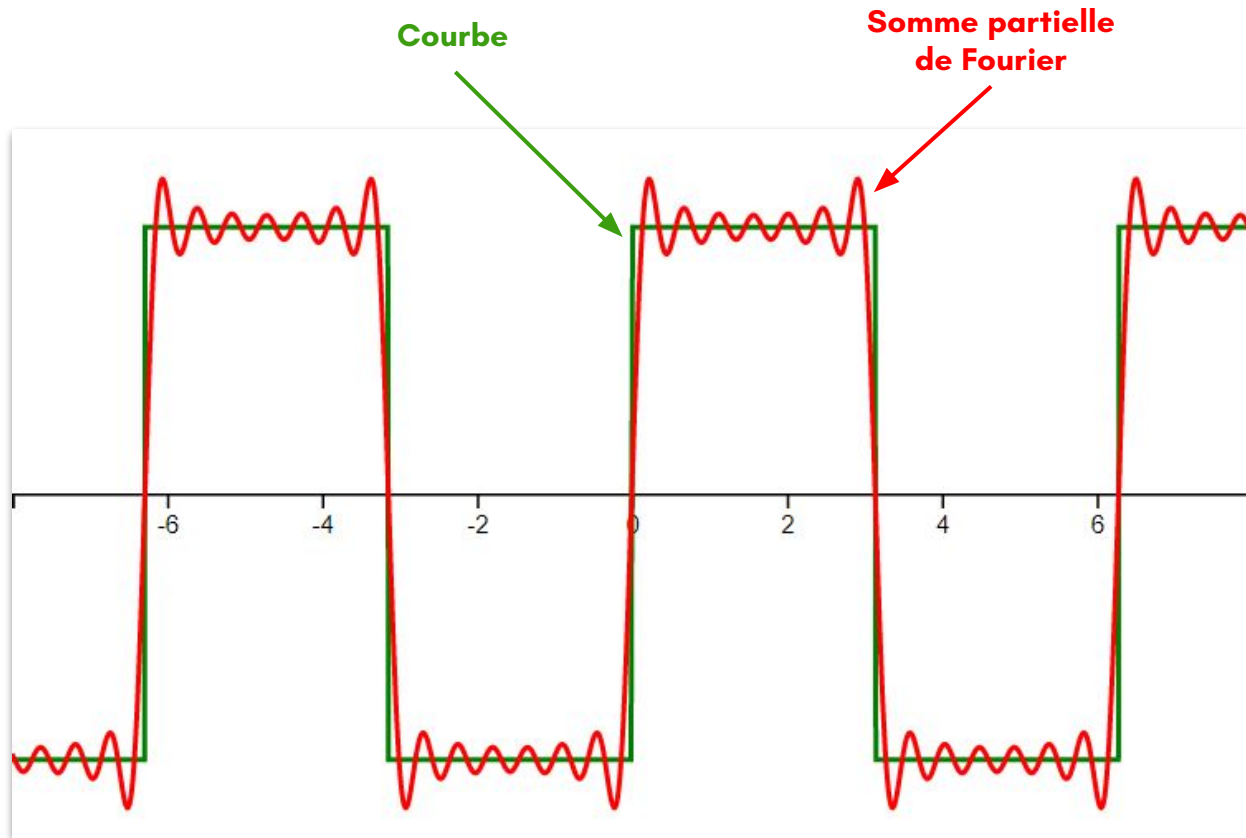
n	re	im
-1181	-0.000671	0.000536
-1180	255.495763	255.49661
-1179	-22.662735	-206.785200
-1178	0.002876	-0.005607
⋮	⋮	⋮
-1	-0.000671	0.000536
0	255.495763	255.49661
1	-22.662735	-206.785200
2	0.002876	-0.005607
⋮	⋮	⋮
1179	-0.000671	0.000536
1180	255.495763	255.49661
1181	-22.662735	-206.785200
1182	0.002876	-0.005607

Nombre de points de l'échantillonnage

Fréquence maximale reconstruite

$$N \geq M$$

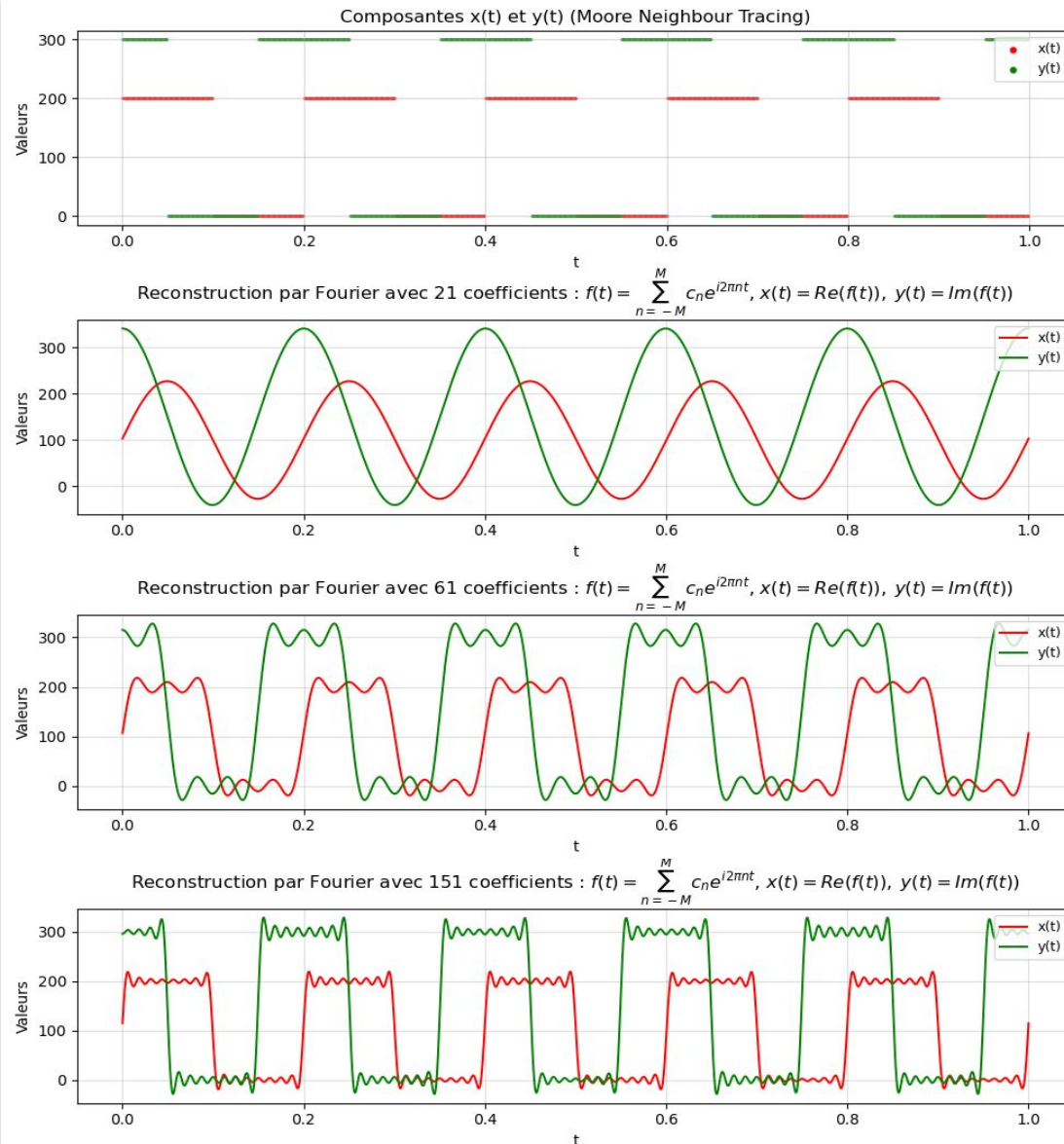
Bruit de Gibbs :



[1]

1: <https://resources.altium.com/fr/p/how-gibbs-phenomenon-produces-measurement-artifacts>

Bruit de Gibbs :

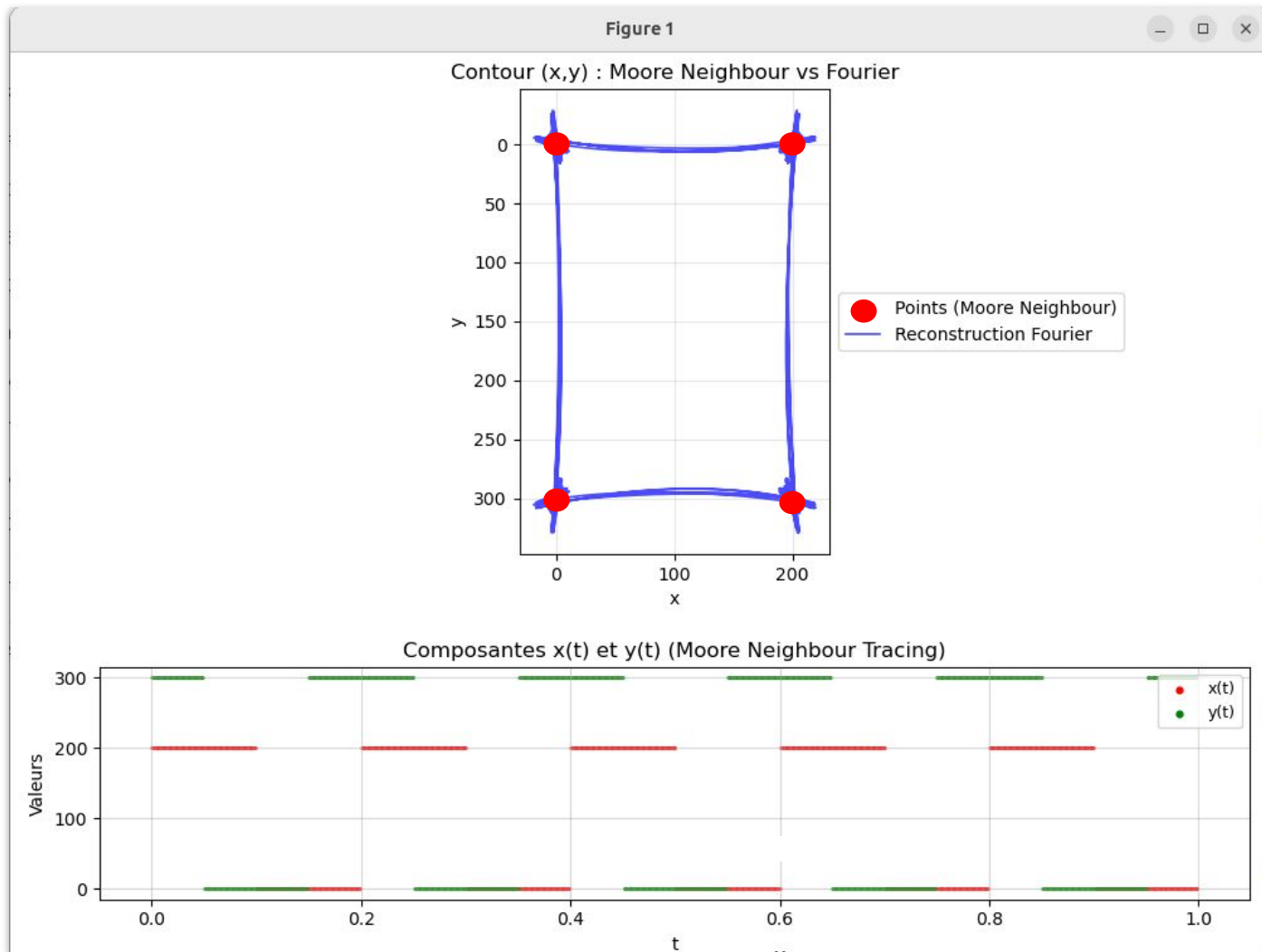


21 coefficients

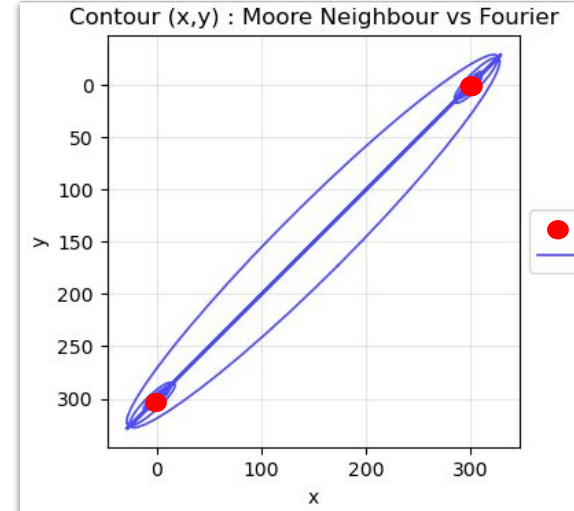
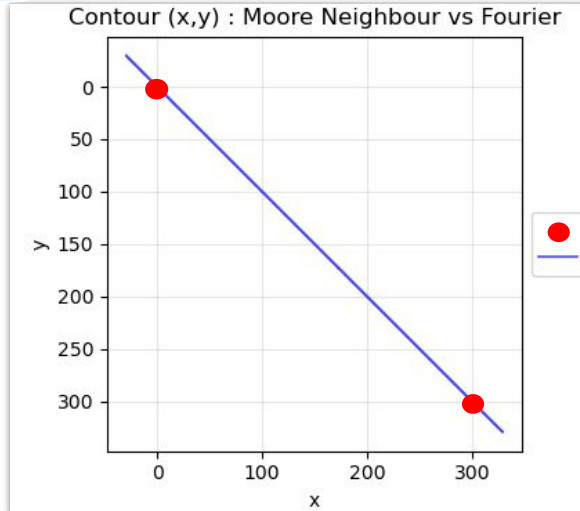
61 coefficients

151 coefficients

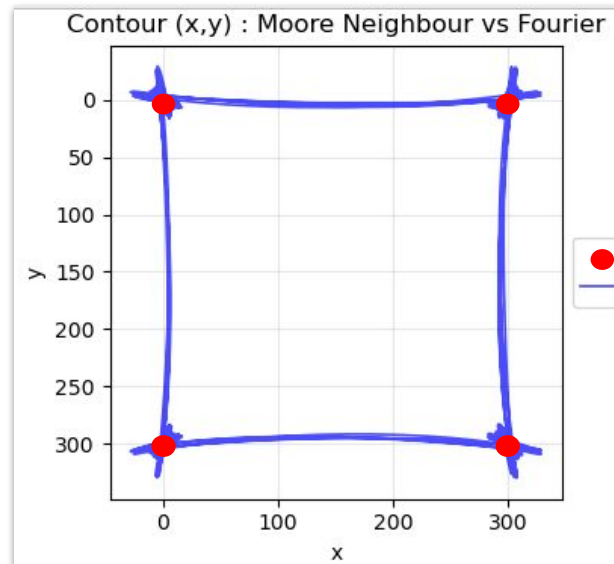
Bruit de Gibbs :



Bruit de Gibbs :



Discontinuité



CONCLUSION

Réussites:

- **Extraction de contour** (voisinage de Moore)
- **Convergence uniforme** (Dirichlet)
- **Reconstruction fidèle et stable avec tri**
- **Phénomène de Gibbs écarté**

Limites et évolutions:

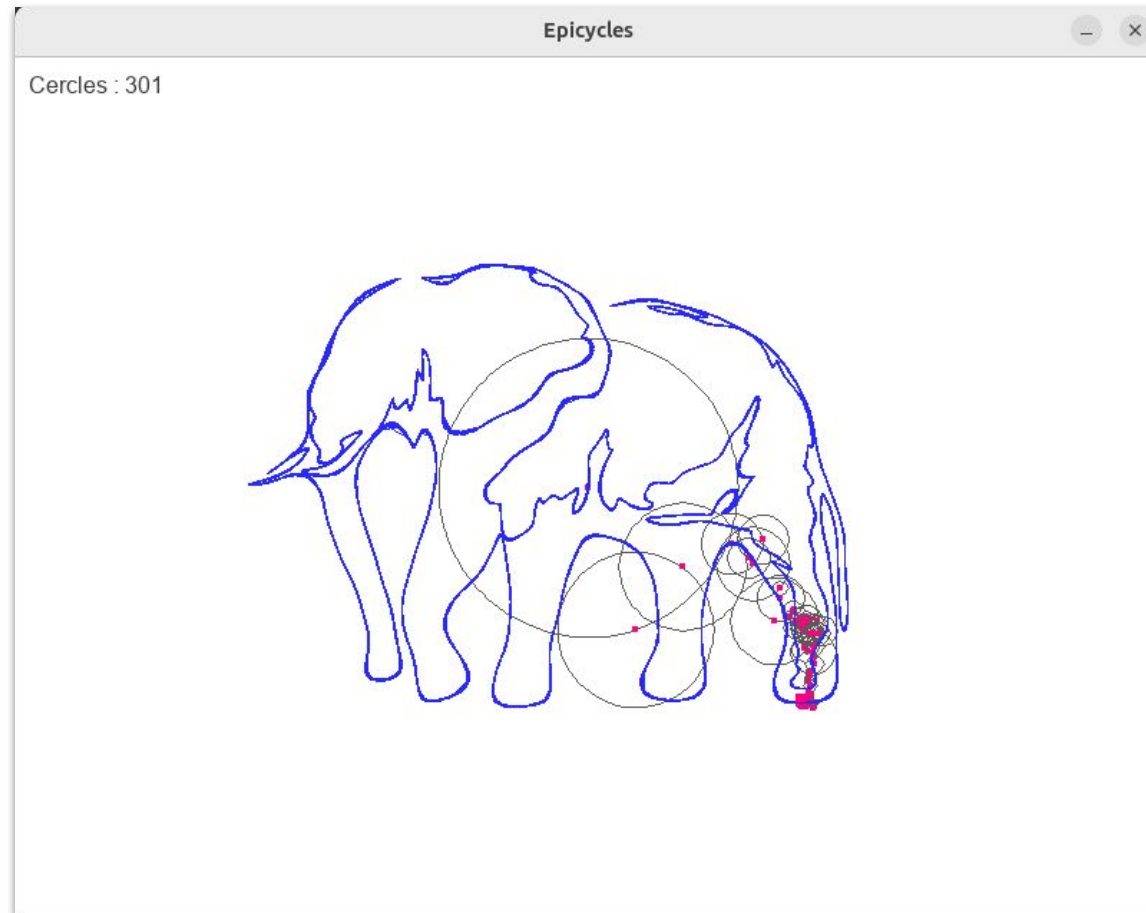
- **Contrainte d'échantillonnage** ($N \geq M$)
- **Bruit inévitable** pour les image sous-résolue, (moins de 100px)
- **Paramétrisation par longueur d'arc** (essais infructueux)

Avec un **contour continue** d'au moins **100 pixels**, une approximation de cette courbe est **traçable** et elle **converge** vers la courbe originale.

CONCLUSION

Merci de votre attention.

Début de l'annexe page suivante



Annexe:

34-38 : Fonctions hors programme utilisées.

39-39 : Démonstration de la convergence de la série de Fourier vers le contour à tracer.

39-44 : Fichier “calcul_eq_parametrique_2.py”

45-66 : Fichier “TIPE_QUE_IMAGE.c”

Fonctions de <stdlib.h>, <string.h>, <stdio.h>:

```
void* realloc(void* ptr, size_t size);
```

- Redimensionne un bloc mémoire alloué.
Peut déplacer les données si nécessaire.

```
int atoi(const char* str);
```

- Convertit une chaîne en entier (sans gestion d'erreurs).

```
int snprintf(char *str, size_t size, const char *format, ...);
```

- Écrit une chaîne formatée dans un tampon `char*`. Le nombre maximal de caractères écrits est limité par la taille spécifiée dans `size`.

Fonctions de <math.h> :

```
double atan2(double y, double x);
```

- Calcule l'angle (en radians) du vecteur (x, y) en tenant compte du quadrant.

```
double hypot(double x, double y);
```

- Calcule $\sqrt{x^2 + y^2}$ de manière stable numériquement.

Fonctions de "lodepng.h" :

```
unsigned lodepng_decode32_file(unsigned char **out, unsigned *w, unsigned *h, const char *filename);
```

- Décode un fichier PNG et retourne une image en format RGBA (4 canaux par pixel).
out : pointeur vers les données image allouées dynamiquement.
w, h : largeur et hauteur de l'image.
Renvoie un code d'erreur (0 si succès).

```
const char* lodepng_error_text(unsigned code);
```

- Retourne une chaîne décrivant l'erreur correspondant au code fourni.

Fonctions de <SDL2/SDL_ttf.h> :

```
int TTF_Init(void);
```

- Initialise le module de rendu de texte.

```
void TTF_Quit(void);
```

- Libère les ressources TTF.

```
TTF_Font* TTF_OpenFont(const char* file, int ptsize);
```

- Charge une police depuis un fichier.

```
void TTF_CloseFont(TTF_Font* font);
```

- Libère une police.

```
SDL_Surface* TTF_RenderText_Blended(TTF_Font* font, const char* text, SDL_Color fg);
```

- Génère une image (surface) contenant le texte avec anti-aliasing.

```
const char* TTF_GetError(void);
```

- Retourne un message d'erreur TTF.

Fonctions de <SDL2/SDL.h> :**Initialisation / destruction**

```
int SDL_Init(Uint32 flags);
```

- Initialise la bibliothèque SDL (ici vidéo avec SDL_INIT_VIDEO).

```
void SDL_Quit(void);
```

- Libère toutes les ressources SDL.

Fenêtre et rendu

```
SDL_Window* SDL_CreateWindow(const char* title, int x, int y, int w, int h, Uint32 flags);
```

- Crée une fenêtre SDL.

```
void SDL_DestroyWindow(SDL_Window* window);
```

- Détruit une fenêtre.

```
SDL_Renderer* SDL_CreateRenderer(SDL_Window* window, int index, Uint32 flags);
```

- Crée un moteur de rendu associé à une fenêtre.

```
void SDL_DestroyRenderer(SDL_Renderer* renderer);
```

- Détruit le renderer.

Fonctions de <SDL2/SDL.h> :**Gestion des événements**

```
int SDL_PollEvent(SDL_Event* event);
```

- Récupère un événement (clavier, souris, fermeture).
Renvoie 1 si un événement est disponible, 0 sinon.

Dessin

```
int SDL_SetRenderDrawColor(SDL_Renderer* renderer, Uint8 r, Uint8 g, Uint8 b, Uint8 a);
```

- Définit la couleur de dessin.

```
int SDL_RenderClear(SDL_Renderer* renderer);
```

- Efface l'écran avec la couleur actuelle.

```
int SDL_RenderDrawLine(SDL_Renderer* renderer, int x1, int y1, int x2, int y2);
```

- Trace une ligne entre deux points.

```
int SDL_RenderFillRect(SDL_Renderer* renderer, const SDL_Rect* rect);
```

- Dessine un rectangle plein.

```
void SDL_RenderPresent(SDL_Renderer* renderer);
```

- Affiche le rendu à l'écran (double buffering).

Fonctions de <SDL2/SDL.h> :**Temps**

`Uint32 SDL_GetTicks(void);`

- Retourne le temps écoulé depuis le lancement du programme (en ms).

`void SDL_Delay(Uint32 ms);`

- Suspend l'exécution pendant ms millisecondes.

Textures / surfaces

`SDL_Texture* SDL_CreateTextureFromSurface(SDL_Renderer* renderer, SDL_Surface* surface);`

- Convertit une surface en texture affichable.

`void SDL_DestroyTexture(SDL_Texture* texture);`

- Libère une texture.

`void SDL_FreeSurface(SDL_Surface* surface);`

- Libère une surface.

`int SDL_RenderCopy(SDL_Renderer* renderer, SDL_Texture* texture, const SDL_Rect* srcrect, const SDL_Rect* dstrect);`

- Copie une texture sur le renderer (affichage).

Erreurs

`const char* SDL_GetError(void);`

- Retourne un message d'erreur SDL.

Démo convergence du contour reconstitué

Contour extrait par l'algorithme de voisins de Moore:

$P_k = (x_k, y_k) \in \mathbb{R}^2$, $k \in [0, \dots, N-1]$ tels que deux pixels successifs soient voisins.

On définit alors $\gamma : [0, 1] \rightarrow \mathbb{C}$, $t_k := \frac{k}{N}$, $\gamma(t_k) = x_k + iy_k$,

Et pour tout $t \in [t_k, t_{k+1}]$, on définit:

$$\gamma(t) = (1 - \lambda(t))P_k + \lambda(t)P_{k+1} \quad \lambda(t) := \frac{t - t_k}{t_{k+1} - t_k}.$$

Ainsi, γ est une fonction affine sur chaque sous-intervalle $[t_k, t_{k+1}]$.

Donc on a bien $\gamma \in C^0([0, 1]) \cap C^1_{\text{morceaux}}([0, 1])$.

On considère la somme partielle $S_M(\gamma)(t) = \sum_{n=-M}^M c_n(\gamma) e^{i2\pi nt}$.

Par le théorème de Dirichlet, on obtient la convergence uniforme :

$\gamma \in C^0([0, 1]) \cap C^1_{\text{morceaux}}([0, 1])$, on a donc $S_M(\gamma)(t) \xrightarrow[M \rightarrow \infty]{CVU} \gamma$

$$\left(\text{ie } \sup_{t \in [0, 1]} |S_M(\gamma)(t) - \gamma(t)| \xrightarrow[M \rightarrow \infty]{} 0 \right)$$

```

1 # -*- coding: utf-8 -*-
2 #
3 #
4 # || TIPE 2025-2026, MPI Lycée Louis Thuillier: Cardot Clément ||
5 #
6 #
7 # ===== Objectif de ce code: =====
8 # Permettre le tracé des composantes x(t) et y(t) du contour correspondant au fichier echantillonnage.csv généré par Calcul_2.c
9 # et la comparaison avec la reconstruction par les coefficients de Fourier calculés par Coeffs_Fourrier.c
10 #
11 # ===== Bibliothèques utilisées: =====
12 # - matplotlib: pour le tracé des graphiques.
13 # - numpy: pour la manipulation efficace des tableaux de données.
14 #
15 # ===== Structure du code: =====
16 # 1. Lecture et stockage des données du fichier CSV.
17 # 2. Création de deux graphiques:
18 #     a. Un graphique pour le contour (x,y) avec les points Moore Neighbour et la reconstruction par Fourier.
19 #     b. Un graphique pour l'évolution de x(t) et y(t) en fonction de t, pour les deux méthodes.
20 # 3. Affichage des courbes.
21
22 print(" || TIPE 2025-2026, MPI Lycée Louis Thuillier: Cardot Clément ||")
23 print("\n → Objectif: Plot des composantes x(t) et y(t) à partir des données du fichier \n echantillonnage.csv et comparaison avec la reconstruction par Fourier.\n")
24
25
26 import matplotlib.pyplot as plt
27 import numpy as np
28
29 # ===== Création d'un créneau =====
30 # N = 1000
31 # periode = 0.2
32 # phase = 0.05
33 #

```



```
34 # with open("echantillonnage.csv", "w") as f:
35 #     f.write("t,x,y\n")
36 #
37 #     for k in range(N):
38 #         t = k / N
39 #
40 #         # Créneau x(t)
41 #         if (t % periode) < (periode / 2):
42 #             x = 200
43 #         else:
44 #             x = 0
45 #
46 #         # Créneau y(t) déphasé
47 #         if ((t + phase) % periode) < (periode / 2):
48 #             y = 300
49 #         else:
50 #             y = 0
51 #
52 #         f.write(f"{t:.3f},{x},{y}\n")
53
54
55 # ===== Lecture echantillonnage.csv (points Moore Neighbour) =====
56 t_points, x_points, y_points = [], [], []
57
58 with open("echantillonnage.csv") as f:
59     next(f) # sauter l'en-tête
60     for line in f:
61         line = line.strip()
62         if not line:
63             continue # ignore ligne vide
64
65         parts = line.split(",")
66         if len(parts) != 3:
67             print("Ligne invalide :", line)
68             continue
```

```
70     t_val, x_val, y_val = parts
71     t_points.append(float(t_val))
72     x_points.append(float(x_val))
73     y_points.append(float(y_val))
74
75 t_points = np.array(t_points)
76 x_points = np.array(x_points)
77 y_points = np.array(y_points)
78 print(" ", len(t_points), "points lus dans echantillonnage.csv.")
79
80
81 # ===== Lecture coeffs_fourier.csv (coefficients) =====
82 n_vals, re_vals, im_vals = [], [], []
83
84 with open("coeffs_fourier.csv") as f:
85     next(f) # sauter l'en-tête
86     for line in f:
87         n, re, im = line.strip().split(",")
88         n_vals.append(int(n))
89         re_vals.append(float(re))
90         im_vals.append(float(im))
91
92 n_vals = np.array(n_vals)
93 c_vals = np.array(re_vals) + 1j*np.array(im_vals)
94
95 nb_coeffs = len(n_vals)
96 print(" ", nb_coeffs, "coefficients lus dans coeffs_fourier.csv.")
97
98
99 # ===== Reconstruction par Fourier =====
100 def reconstruire(t, n_vals, c_vals):
101     """Calcule f(t) = somme c_n * exp(i*2pi*n*t)."""
102     somme = np.zeros_like(t, dtype=complex)
103     for n, c in zip(n_vals, c_vals):
104         somme += c * np.exp(1j * 2*np.pi * n * t)
```

```

105     return somme
106
107 t_reconstruit = np.linspace(0, 1, 2000)
108 f_t = reconstruire(t_reconstruit, n_vals, c_vals)
109 x_reconstruit = f_t.real
110 y_reconstruit = f_t.imag
111 print("  Reconstruction par Fourier effectuée.")
112
113
114 # ===== Tracés avec proportions adaptées =====
115 fig, axes = plt.subplots(
116     3, 1, figsize=(10, 12),
117     gridspec_kw={"height_ratios": [2, 1, 1]} # plot (x,y) plus grand
118 )
119
120 # (1) Courbe des composantes (x,y) : Fourier vs Moore Neighbour
121 axes[0].scatter(x_points, y_points, s=0.7, color="gray", label="Points (Moore Neighbour)")
122 axes[0].plot(x_reconstruit, y_reconstruit, color="blue", linewidth=1.2, alpha=0.7, label="Reconstruction Fourier")
123 axes[0].set_aspect("equal")
124 axes[0].invert_yaxis()
125 axes[0].set_xlabel("x")
126 axes[0].set_ylabel("y")
127 axes[0].set_title("Contour (x,y) : Moore Neighbour vs Fourier")
128 axes[0].legend(loc="center left", bbox_to_anchor=(1, 0.5))
129 axes[0].grid(True, alpha=0.3)
130
131 # (2) Courbes x(t), y(t) Moore Neighbour
132 axes[1].scatter(t_points, x_points, s=0.5, c='red', label='x(t)')
133 axes[1].scatter(t_points, y_points, s=0.5, c='green', label='y(t)')
134 axes[1].set_title("Composantes x(t) et y(t) (Moore Neighbour Tracing)")
135 axes[1].set_xlabel("t")
136 axes[1].set_ylabel("Valeurs")
137 axes[1].legend(fontsize=9, markerscale=5, loc="upper right")
138 axes[1].grid(True, alpha=0.5)
139

```

```
140 # (3) Courbes x(t), y(t) par Fourier
141 axes[2].plot(t_reconstruit, x_reconstruit, c='red', linewidth=1.4, label='x(t)')
142 axes[2].plot(t_reconstruit, y_reconstruit, c='green', linewidth=1.4, label='y(t)')
143 axes[2].set_title(rf"Reconstruction par Fourier avec {nb_coeffs} coefficients : "
144                  r"$f(t)=\sum_{n=-M}^M c_n e^{i 2\pi n t}$, "
145                  r"$x(t)=\text{Re}(f(t)), \text{ } y(t)=\text{Im}(f(t))$")
146 axes[2].set_xlabel("t")
147 axes[2].set_ylabel("Valeurs")
148 axes[2].legend(fontsize=9, loc="upper right")
149 axes[2].grid(True, alpha=0.4)
150
151 plt.tight_layout()
152 print(" Affichage en cours.")
153 plt.show()
154
155 print("\n Fermeture du programme.\n")
```

```

1  /*
2
3  [
4      TIPE 2025-2026, MPI Lycée Louis Thuillier: Cardot Clément
5      Objectif: Animation SDL d'épicycles à partir de Fourier
6  ]
7
8  ===== Objectif de ce code: =====
9
10 ETAPE 1.
11 -Permet de lire une image PNG (passé en paramètre) grâce à lodepng et en extraire le contour par l'algorithme Moore Neighbor Tracing.
12 -Enregistre le résultat dans le fichier echantillonnage.csv (t, x(t), y(t)) pour t dans [0;1].
13
14 ETAPE 2.
15 -calcule les 2M + 1 coefficients de Fourier complexes associés (M donné en paramètre).
16 -En sortie, le programme génère le fichier coeffs_fourier.csv contenant (n, Re(c_n), Im(c_n)) pour tout n dans [-M ; M].
17
18 ETAPE 3.
19 -Permet de tracer des épicycles animés à partir de coefficients de Fourier, avec une interface graphique réalisée avec SDL2.
20 -Affiche en temps réel les paramètres du tracé (nombre de cercles, t, scale) et permet de mettre en pause l'animation avec la barre d'espace.
21
22 ===== Bibliothèques utilisées: =====
23 - lodepng pour la lecture des fichiers PNG.
24   JOINDRE: lodepng.h et lodepng.c (lodepng.cpp à renommer), fichiers sur https://lodev.org/lodepng/
25 - SDL2 pour le rendu graphique et la gestion de la fenêtre.
26   NECESSITE SDL2 (Simple DirectMedia Layer) - https://www.libsdl.org/
27   (aussi SDL2_ttf pour le texte)
28 - math.h pour les fonctions trigonométriques.
29
30 ===== Modifications: =====
31 AVANT: ./Dessin 0 image.png 50
32
33 L'objectif est donc de prendre seulement une image en argument, contenant le contour à tracer et de faire le choix de M automatiquement.
34 L'approximation des intégrales pour le calcul des coefficients de Fourier oblige à respecter  $N > M$  (sinon  $c_{\{n+N\}} = c_n$ ), donc je fixe  $M:=N/10$ .
35 Plus possible de tracer les epicycles classiques.
36
37 MAINTENANT: ./Dessin image.png      => tracé avec  $M:=N/10$ 

```

```

36      ./Dessin image.png M  => tracé avec M donné en paramètres
37
38  ===== Compilation: =====
39  gcc -o Dessin TIPE_QUE_IMAGE.c lodepng.c -lSDL2 -lSDL2_ttf -lm && ./Dessin image.png
40  */
41
42
43  /*
44  ||
45  ||          ETAPE 1:
46  ||  Lire le .png avec lodepng, algorithme des voisins de Moore
47  ||  Résultat en format (t, x(t), y(t))
48  ||
49  */
50
51  // ===== include, définitions et types =====
52  #include <stdio.h>
53  #include <stdlib.h>
54  #include <string.h>
55  #include <stdbool.h>
56
57  #include <SDL2/SDL_ttf.h>
58  #include <math.h>
59  #include "lodepng.h"
60  #include <SDL2/SDL.h>
61  #define PI 3.14159265358979323846
62
63  typedef struct {
64      double x, y;
65  } Point_grille;
66
67
68  // ===== Lecture du PNG =====
69  unsigned char* decoder_png_rgba(const char* nom_fichier, unsigned* largeur, unsigned* hauteur) {
70      unsigned char* image = NULL;

```

```
71 unsigned erreur = lodepng_decode32_file(&image, largeur, hauteur, nom_fichier);
72 if (erreur) {
73     printf("Erreur %u: %s\n", erreur, lodepng_error_text(erreur));
74     return NULL;
75 }
76 return image;
77 }
78
79
80 // ===== Traitement de l'image =====
81
82 unsigned char* convertir_en_N_et_B(unsigned char* rgba, unsigned largeur, unsigned hauteur) {
83     unsigned char* gris = malloc(largeur * hauteur);
84     for (unsigned y = 0; y < hauteur; y++) {
85         for (unsigned x = 0; x < largeur; x++) {
86             unsigned indice = 4 * (y * largeur + x);
87             unsigned char r = rgba[indice];
88             unsigned char g = rgba[indice + 1];
89             unsigned char b = rgba[indice + 2];
90             unsigned char a = rgba[indice + 3];
91             if (a == 0) gris[y*largeur + x] = 255; // Alpha 0 = blanc
92             else gris[y*largeur + x] = (r + g + b)/3;
93         }
94     }
95     return gris;
96 }
97
98 void binariser(unsigned char* gris, unsigned largeur, unsigned hauteur, unsigned int seuil) {
99     for (unsigned i = 0; i < largeur*hauteur; i++){
100         if (gris[i] > seuil)
101             gris[i] = 255;
102         else
103             gris[i] = 0;
104     }
105 }
```

```
107 void inverser_si_necessaire(unsigned char* binaire, unsigned largeur, unsigned hauteur) {
108     int noir = 0, blanc = 0;
109     for (unsigned i = 0; i < largeur*hauteur; i++) {
110         if (binaire[i] == 0)
111             noir++;
112         else
113             blanc++;
114     }
115     if (blanc < noir) { // la forme est probablement blanche → inverser
116         for (unsigned i = 0; i < largeur*hauteur; i++){
117             binaire[i] = 255 - binaire[i];
118         }
119     }
120 }
121
122
123 // ===== Algorithme du voisinage de Moore =====
124
125 int voisins_de_moore(unsigned char* binaire, unsigned largeur, unsigned hauteur, Point_grille** contour, int* max_l, int* max_h) {
126     int capacite = 1000, compteur = 0;
127     *contour = malloc(capacite * sizeof(Point_grille));
128
129     int x_depart = -1, y_depart = -1;
130     for (unsigned y = 1; y < hauteur-1 && x_depart == -1; y++){
131         for (unsigned x = 1; x < largeur-1; x++){
132             if (binaire[y*largeur + x] == 0){
133                 x_depart = x;
134                 y_depart = y;
135                 break;
136             }
137         }
138     }
139     if (x_depart == -1) return 0;
140
141     int direction[8][2] = { {0,-1}, {1,-1}, {1,0}, {1,1}, {0,1}, {-1,1}, {-1,0}, {-1,-1} };
```



```
142 int x_actuel = x_depart, y_actuel = y_depart;
143 int x_precedent = x_depart, y_precedent = y_depart - 1;
144
145 int xmin = x_depart, xmax = x_depart, ymin = y_depart, ymax = y_depart;
146
147 do {
148     if (compteur >= capacite) {
149         capacite *= 2;
150         *contour = realloc(*contour, capacite * sizeof(Point_grille));
151     }
152     (*contour)[compteur++] = (Point_grille){x_actuel, y_actuel};
153
154     if (x_actuel < xmin) xmin = x_actuel;
155     if (x_actuel > xmax) xmax = x_actuel;
156     if (y_actuel < ymin) ymin = y_actuel;
157     if (y_actuel > ymax) ymax = y_actuel;
158
159     int trouve = 0;
160     int direction_depart = -1;
161     for (int i = 0; i < 8; i++){
162         if (x_actuel + direction[i][0] == x_precedent && y_actuel + direction[i][1] == y_precedent){
163             direction_depart = (i + 1) % 8;
164             break;
165         }
166     }
167     for (int k = 0; k < 8; k++) {
168         int i = (direction_depart + k) % 8;
169         int x_suivant = x_actuel + direction[i][0];
170         int y_suivant = y_actuel + direction[i][1];
171         if (binaire[y_suivant*largeur + x_suivant] == 0) {
172             x_precedent = x_actuel;
173             y_precedent = y_actuel;
174             x_actuel = x_suivant;
175             y_actuel = y_suivant;
176             trouve = 1;
```

```

177         break;
178     }
179 }
180 if (!trouve) break;
181
182 } while (x_actuel != x_depart || y_actuel != y_depart);
183
184 *max_l = xmax - xmin;
185 *max_h = ymax - ymin;
186
187 return compteur;
188 }
189
190
191 void calculer_t(Point_grille* contour, int nb_points, double* valeurs_t) {
192     for (int i = 0; i < nb_points; i++){
193         valeurs_t[i] = (double)i / nb_points;
194     }
195 }
196
197
198 /*
199     ||
200     ||          ETAPE 2:
201     ||          Calcul coeffs de Fourier par intégrale de Riemann
202     ||          Résultat en format (n, Re(c_n), Im(c_n))
203     ||
204 */
205
206 // ===== include, def et types =====
207
208 typedef struct Complexe {
209     double re, im;
210 } Complexe;
211

```

```
212 typedef struct Point_parametrique {
213     double t, x, y;
214 } Point_parametrique;
215
216 typedef struct Coefficient {
217     int n;
218     Complexe coeff;
219 } Coefficient;
220
221
222 // ===== Primitives sur les complexes =====
223
224 Complexe complexe_ajout(Complexe a, Complexe b) {
225     Complexe r;
226     r.re = a.re + b.re;
227     r.im = a.im + b.im;
228     return r;
229 }
230
231 Complexe complexe_produit(Complexe a, Complexe b) {
232     Complexe r;
233     r.re = a.re * b.re - a.im * b.im;
234     r.im = a.re * b.im + a.im * b.re;
235     return r;
236 }
237
238 Complexe complexe_exp(double module, double theta) {
239     Complexe r;
240     r.re = module * cos(theta);
241     r.im = module * sin(theta);
242     return r;
243 }
244
245
246 // ===== Calcul des coefficients =====
```

```
248 Complexe calculer_un_coeff(Point_parametrique* pts, int N, int n) {
249     Complexe somme;
250     somme.re = 0.0;
251     somme.im = 0.0;
252     double T = 1.0; // suppose t dans [0,1]
253     double omega = 2*PI/T;
254
255     for (int k = 0; k < N; k++) {
256         Complexe ftk;
257         ftk.re = pts[k].x;
258         ftk.im = pts[k].y;
259         double theta = - n * omega * pts[k].t;
260         Complexe expterm = complexe_exp(1.0, theta);
261
262         Complexe prod = complexe_produit(ftk, expterm);
263
264         somme.re += prod.re;
265         somme.im += prod.im;
266     }
267
268     somme.re /= N;
269     somme.im /= N;
270     return somme;
271 }
272
273 void calculer_tous_les_coeffs(Point_parametrique* pts, int N, int M, Coefficient* coeffs) {
274     int idx = 0;
275     for (int n = -M; n <= M; n++) {
276         Complexe c = calculer_un_coeff(pts, N, n);
277         coeffs[idx].n = n;
278         coeffs[idx].coeff = c;
279         idx++;
280     }
281 }
282
```

```

285  /*
286
287      ETAPE 3:
288      Dessin avec SDL, implémentation de l'interface
289      def du tableau cercles[] à partir des coeffs de fourier
290
291  */
292
293
294  // ===== include, def et types =====
295  const int LARGEUR_FENETRE = 800;
296  const int HAUTEUR_FENETRE = 600;
297  //const int TAILLE_MAX_CHEMIN = 500; // Voir la ligne disparaître
298  const int TAILLE_MAX_CHEMIN = 15000; // Voir la ligne tout le temps
299  double SPEED = 1;
300
301
302  // ===== Struct Cercle =====
303  typedef struct {
304      double cx, cy;    // centre (double maintenant)
305      double rayon;     // rayon (Module du coeff de Fourier)
306      double angle;     // phase initiale (argument du coeff de Fourier)
307      double vitesse;   // vitesse angulaire (2*PI*n pour le n-ieme coeff, en rad/s)
308      int sens;         // sens de rotation : +1 ou -1 (dépend du signe de n)
309  } Cercle;
310
311
312  // ===== SDL Utils =====
313
314  bool init_SDL(SDL_Window **win, SDL_Renderer **ren) {
315      if (SDL_Init(SDL_INIT_VIDEO) < 0) {
316          printf("Erreur SDL: %s\n", SDL_GetError());
317          return false;
318      }
319

```

```
321     if (TTF_Init() == -1) {
322         printf("Erreur TTF_Init: %s\n", TTF_GetError());
323         return false;
324     }
325
326     *win = SDL_CreateWindow("Epicycles", SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED,
327                             LARGEUR_FENETRE, HAUTEUR_FENETRE, SDL_WINDOW_SHOWN);
328
329     if (!*win) {
330         printf("Erreur creation fenetre: %s\n", SDL_GetError());
331         return false;
332     }
333
334     *ren = SDL_CreateRenderer(*win, -1, SDL_RENDERER_ACCELERATED);
335     if (!*ren) {
336         printf("Erreur creation renderer: %s\n", SDL_GetError());
337         return false;
338     }
339
340     return true;
341 }
342
343 void nettoyage_SDL(SDL_Window *win, SDL_Renderer *ren) {
344     SDL_DestroyRenderer(ren);
345     SDL_DestroyWindow(win);
346     TTF_Quit();
347     SDL_Quit();
348 }
349
350 // ===== Dessin =====
351
352 void clear_ecran(SDL_Renderer *ren, int r, int g, int b) {
353     SDL_SetRenderDrawColor(ren, r, g, b, 255);
354     SDL_RenderClear(ren);
355 }
```

```
357 void dessiner_cercle(SDL_Renderer *ren, int cx, int cy, int rayon) {
358     SDL_SetRenderDrawColor(ren, 90, 90, 90, 255);
359     const int NB_SEGMENTS = (2 * PI * rayon) / 5;
360     double theta1 = 0;
361     double theta2 = (double)1 / NB_SEGMENTS * 2 * PI;
362     for (int i = 0; i < NB_SEGMENTS; i++) {
363         int x1 = cx + (int)(rayon * cos(theta1));
364         int y1 = cy + (int)(rayon * sin(theta1));
365         int x2 = cx + (int)(rayon * cos(theta2));
366         int y2 = cy + (int)(rayon * sin(theta2));
367         SDL_RenderDrawLine(ren, x1, y1, x2, y2);
368         theta1 = theta2;
369         theta2 = theta1 + (double)1 / NB_SEGMENTS * 2 * PI;
370     }
371 }
372
373 void dessiner_point(SDL_Renderer *ren, Point_grille p, int taille, int r, int g, int b) {
374     SDL_SetRenderDrawColor(ren, r, g, b, 100);
375     SDL_Rect rect = {(int)(p.x - taille/2), (int)(p.y - taille/2), taille, taille};
376     SDL_RenderFillRect(ren, &rect);
377 }
378
379 void dessiner_chemin(SDL_Renderer *ren, Point_grille chemin[], int longueur) {
380     SDL_SetRenderDrawColor(ren, 40, 40, 255, 255);
381     for (int i = 1; i < longueur; i++) {
382         SDL_RenderDrawLine(ren, (int)chemin[i-1].x, (int)chemin[i-1].y, (int)chemin[i].x, (int)chemin[i].y);
383     }
384 }
385
386 void afficher_texte(SDL_Renderer *ren, const char *texte, int x, int y, SDL_Color couleur, int taille) {
387     TTF_Font *font = TTF_OpenFont("arial.ttf", taille);
388     if (!font) {
389         printf("Erreur font : %s\n", TTF_GetError());
390         return;
391     }
```

```
393 SDL_Surface *surface = TTF_RenderText_Blended(font, texte, couleur);
394 SDL_Texture *texture = SDL_CreateTextureFromSurface(ren, surface);
395
396 SDL_Rect rect = {x, y, surface->w, surface->h};
397 SDL_RenderCopy(ren, texture, NULL, &rect);
398
399 SDL_FreeSurface(surface);
400 SDL_DestroyTexture(texture);
401 TTF_CloseFont(font);
402 }
403
404 // ===== Logique =====
405
406 void update_cercle(Cercle *c, double dt) {
407     c->angle += c->sens * c->vitesse * SPEED * dt;
408     if (c->angle > 2 * PI) c->angle -= 2 * PI;
409     if (c->angle < 0) c->angle += 2 * PI;
410 }
411
412 Point_grille point_sur_cercle(Cercle c) {
413     Point_grille p;
414     p.x = c.cx + c.rayon * cos(c.angle);
415     p.y = c.cy + c.rayon * sin(c.angle);
416     return p;
417 }
418
419
420
421 // ===== Affichage du terminal =====
422 const int LARGEUR_TEXTE = 70; // largeur intérieure du cadre (à ajuster si besoin)
423
424 void afficher_ligne(const char *texte, int largeur_texte){
425     printf(" || %-*s ||\n", largeur_texte-3, texte); //-3 car "%-*s" est considéré comme 1 donc decalage de 3...
426 }
427
```



```
428 void afficher_bord_haut()
429 {
430     printf("\n\n ⌌");
431     for (int i = 0; i < LARGEUR_TEXTE; i++)
432         printf("=");
433     printf("⌌\n");
434 }
435
436 void afficher_bord_bas()
437 {
438     printf(" ⌌");
439     for (int i = 0; i < LARGEUR_TEXTE; i++)
440         printf("=");
441     printf("⌌\n\n");
442 }
443 void afficher_ligne_vide(){
444     afficher_ligne("", LARGEUR_TEXTE);
445 }
446 void afficher_titre(){
447     int espace_g = (LARGEUR_TEXTE - 61)/2;
448     int espace_d = LARGEUR_TEXTE - 61 -espace_g;
449     printf(" ⌌");
450     for(int i = 0; i < espace_g; i++){printf(" ");}
451     printf(" ⌌");
452     for(int i = 0; i < 59; i++){printf("=");}
453     printf("⌌");
454     for(int i = 0; i < espace_d; i++){printf(" ");}
455     printf("⌌\n");
456
457     printf(" ⌌");
458     for(int i = 0; i < espace_g; i++){printf(" ");}
459     printf("⌌ TIPE 2025-2026, MPI* Lycée Louis Thuillier:Cardot Clément ⌌");
460     for(int i = 0; i < espace_d; i++){printf(" ");}
461     printf("⌌\n");
462 }
```

```

463     printf(" ||");
464     for(int i = 0; i < espace_g; i++){printf(" ");}
465     printf("L");
466     for(int i = 0; i < 59; i++){printf("=");}
467     printf("J");
468     for(int i = 0; i < espace_d; i++){printf(" ");}
469     printf(" ||\n");
470 }
471
472 void afficher_erreur_usage()
473 {
474     afficher_ligne(" - image.png : image contenant la forme a traiter", LARGEUR_TEXTE);
475     afficher_ligne("          sur fond blanc", LARGEUR_TEXTE);
476     afficher_ligne_vide();
477     afficher_ligne(" Facultatif: ", LARGEUR_TEXTE);
478     afficher_ligne(" - M : ordre des coefficients de Fourier (2M+1 coeffs)", LARGEUR_TEXTE);
479     afficher_ligne("          M = N/10 si pas spécifié", LARGEUR_TEXTE+2);
480 }
481
482
483 // ===== Boucle principale =====
484
485 void main_loop(SDL_Renderer *ren, Coefficient *coeffs, int M, float SCALE) {
486
487     int nb_cercles = 0;
488     Cercle *cercles = NULL;
489
490     SPEED = 0.15;
491     nb_cercles = 2 * M + 1;
492     cercles = malloc(nb_cercles * sizeof(Cercle));
493     cercles[0] = (Cercle){LARGEUR_FENETRE / 2, HAUTEUR_FENETRE / 2, 0, 0, 0, 1};
494     int num_cercle = 1;
495     for (int k = 1; k <= M; k++) {
496         // -k
497         Coefficient coeff_neg = coeffs[M + k];

```

```
498     cercles[num_cercle++] = (Cercle){
499         0, 0, SCALE * hypot(coeff_neg.coeff.re, coeff_neg.coeff.im), atan2(coeff_neg.coeff.im, coeff_neg.coeff.re), k * 2 * PI, -1
500     };
501     // +k
502     Coefficient coeff_pos = coeffs[M - k];
503     cercles[num_cercle++] = (Cercle){
504         0, 0, SCALE * hypot(coeff_pos.coeff.re, coeff_pos.coeff.im), atan2(coeff_pos.coeff.im, coeff_pos.coeff.re), k * 2 * PI, +1
505     };
506 }
507
508 SDL_Event epicycles;
509 bool quitter = false;
510 bool en_pause = false;
511 Uint32 last_time = SDL_GetTicks();
512 double t = 0.0;
513
514 Point_grille chemin[TAILLE_MAX_CHEMIN];
515 int longueur_chemin = 0;
516
517 // Boucle principal de traçage
518 while (!quitter) {
519     while (SDL_PollEvent(&epicycles)) { //Action pause
520         if (epicycles.type == SDL_QUIT)
521             quitter = true;
522
523         if (epicycles.type == SDL_KEYDOWN) {
524             switch (epicycles.key.keysym.sym) {
525                 case SDLK_ESCAPE:
526                     quitter = true;
527                     break;
528                 case SDLK_SPACE:
529                     en_pause = !en_pause;
530                     break;
531             }
532         }
533     }
```

```
533     }
534
535     //Calcul temps écoulé
536     Uint32 current_time = SDL_GetTicks();
537     double dt = (current_time - last_time) / 1000.0;
538     last_time = current_time;
539
540     if(!en_pause){
541         t += dt * SPEED;
542         if(t > 1.0){
543             t -= 1.0;
544         }
545
546         Point_grille centre_courant = {cercles[0].cx, cercles[0].cy};
547
548         for (int i = 0; i < nb_cercles; i++) {
549             cercles[i].cx = centre_courant.x;
550             cercles[i].cy = centre_courant.y;
551             update_cercle(&cercles[i], dt);
552             centre_courant = point_sur_cercle(cercles[i]);
553         }
554
555         if (longueur_chemin < TAILLE_MAX_CHEMIN)
556             chemin[longueur_chemin++] = centre_courant;
557         else {
558             memmove(chemin, chemin + 1, (TAILLE_MAX_CHEMIN - 1) * sizeof(Point_grille));
559             chemin[TAILLE_MAX_CHEMIN - 1] = centre_courant;
560         }
561     }
562
563     // Rendu graphique
564     clear_ecran(ren, 255, 255, 255);
565
566     Point_grille centre_courant = {cercles[0].cx, cercles[0].cy};
567
```

```
568     for (int i = 0; i < nb_cercles - 1; i++) {
569         dessiner_cercle(ren, cercles[i].cx, cercles[i].cy, cercles[i].rayon);
570         Point_grille p = point_sur_cercle(cercles[i]);
571         dessiner_point(ren, p, 4, 255, 0, 128);
572         centre_courant = p;
573     }
574
575     dessiner_cercle(ren, cercles[nb_cercles - 1].cx, cercles[nb_cercles - 1].cy,
576                   cercles[nb_cercles - 1].rayon);
577     Point_grille p = point_sur_cercle(cercles[nb_cercles - 1]);
578     dessiner_point(ren, p, 7, 255, 0, 128);
579     centre_courant = p;
580
581     dessiner_chemin(ren, chemin, longueur_chemin);
582
583     SDL_Color gris = {60, 60, 60, 255};
584     SDL_Color vert = {0, 255, 100, 255};
585     SDL_Color rouge = {220, 40, 40, 255};
586
587     char buffer[128];
588     snprintf(buffer, sizeof(buffer), "Cercles : %d", nb_cercles);
589     afficher_texte(ren, buffer, 10, 10, gris, 18);
590
591     snprintf(buffer, sizeof(buffer), "SCALE : %.2f", SCALE);
592     afficher_texte(ren, buffer, 10, 55, gris, 16);
593
594     snprintf(buffer, sizeof(buffer), "t : %.3f", t);
595     afficher_texte(ren, buffer, 10, 75, vert, 16);
596
597     if(en_pause){
598         afficher_texte(ren, " PAUSE ", LARGEUR_FENETRE/2 - 45, HAUTEUR_FENETRE - 50, rouge, 25);
599     }
600
601     SDL_RenderPresent(ren);
602     SDL_Delay(10);
```

```
603     }
604
605     free(cercles);
606 }
607
608
609
610
611 /*
612  ||
613  ||             MAIN
614  ||         Traçage de la forme
615  ||
616  ||
617  */
618 */
619
620 int main(int argc, char** argv){
621     bool csv = true; // Permet de choisir si on veut générer les fichiers csv ou non
622     char buffer[200]; //Gerer l'affichage des espaces
623
624     afficher_bord_haut();
625     afficher_titre();
626     afficher_ligne_vide();
627
628     if (argc < 2){
629         snprintf(buffer, sizeof(buffer), "ERREUR: usage: %s image.png", argv[0]);
630         afficher_ligne(buffer, LARGEUR_TEXTE);
631         afficher_ligne_vide();
632         afficher_ligne_vide();
633         afficher_erreur_usage();
634         afficher_ligne_vide();
635         afficher_ligne(" Fermeture du programme.", LARGEUR_TEXTE);
636         afficher_bord_bas();
637         return 1;
638     }
```

```
638 }
639
640 // =====
641 //           ÉTAPE 1: on traite l'image, calcul du contour
642 // =====
643
644 const char* image_file = argv[1];
645
646 snprintf(buffer, sizeof(buffer), "→ Étape 1 : Extraction du contour de l'image %s", image_file);
647 afficher_ligne(buffer, LARGEUR_TEXTE+3); //+3 car la flèche est un caractère qui compte pour 2, %s aussi et l'accent aussi...
648
649 unsigned int largeur, hauteur;
650 unsigned char* rgba = decoder_png_rgba(image_file, &largeur, &hauteur);
651 if (!rgba){
652     afficher_ligne("  ERREUR PNG, RGBA FALSE", LARGEUR_TEXTE);
653     afficher_bord_bas();
654     return 1;
655 }
656
657 unsigned char* gris = convertir_en_N_et_B(rgba, largeur, hauteur);
658 free(rgba);
659
660 binariser(gris, largeur, hauteur, 128);
661 inverser_si_necessaire(gris, largeur, hauteur);
662
663 Point_grille* contour = NULL;
664
665 int max_l, max_h;
666 int nb_points = voisins_de_moore(gris, largeur, hauteur, &contour, &max_l, &max_h);
667
668 double* valeurs_t = malloc(nb_points * sizeof(double));
669 calculer_t(contour, nb_points, valeurs_t);
670 snprintf(buffer, sizeof(buffer), "  Nombre de points du contour : %d", nb_points);
671 afficher_ligne(buffer, LARGEUR_TEXTE);
672
```

```

673 // Écrire le fichier echantillonage.csv
674 if(csv){
675     FILE* echantillonage_csv = fopen("echantillonage.csv", "w");
676     fprintf(echantillonage_csv, "t,x,y\n");
677     for (int i = 0; i < nb_points; i++){
678         fprintf(echantillonage_csv, "%f,%d,%d\n", valeurs_t[i], (int)(contour[i].x), (int)(contour[i].y));
679     }
680     fclose(echantillonage_csv);
681     afficher_ligne("    Fichier echantillonage.csv écrit.", LARGEUR_TEXTE+1); // "é" compte pour 2 (Pourquoi ?! aucune idée...)
682 }
683
684 free(gris);
685
686 // =====
687 //                               ÉTAPE 2: Calcul des coefficients de Fourier
688 // =====
689
690 int M;
691 if (argc > 2){
692     M = atoi(argv[2]);
693 }
694 else{
695     M = nb_points / 10;    // choix automatique
696 }
697 afficher_ligne_vide();
698 snprintf(buffer, sizeof(buffer), "→ Étape 2 : Calcul des coefficients de Fourier (M=%d)", M);
699 afficher_ligne(buffer, LARGEUR_TEXTE+3); //idem, fleche et accent
700
701
702 Point_parametrique* points = malloc(nb_points * sizeof(Point_parametrique));
703 for (int i = 0; i < nb_points; i++){
704     points[i].t = (double)i / nb_points;
705     points[i].x = contour[i].x;
706     points[i].y = contour[i].y;
707 }

```



```

708 int total = 2*M + 1;
709 Coefficient* coeffs = malloc(total * sizeof(Coefficient));
710 calculer_tous_les_coeffs(points, nb_points, M, coeffs);
711
712 // Écrire le fichier coeffs_fourier.csv
713 if(csv){
714     FILE* coeffs_fourier_csv = fopen("coeffs_fourier.csv", "w");
715     fprintf(coeffs_fourier_csv, "n,re,im\n");
716     for (int i = 0; i < 2*M+1; i++) {
717         fprintf(coeffs_fourier_csv, "%d,%lf,%lf\n", coeffs[i].n, coeffs[i].coeff.re, coeffs[i].coeff.im);
718     }
719     fclose(coeffs_fourier_csv);
720     afficher_ligne("    Fichier coeffs_fourier.csv écrit.", LARGEUR_TEXTE+1); // "é" compte pour 2
721 }
722
723 free(points);
724 free(contour);
725
726 // =====
727 //          ETAPE 3: Dessin des épicycles à l'aide des coeffs
728 // =====
729
730 afficher_ligne_vide();
731 afficher_ligne("→ Étape 3 : Tracé de la forme avec des epicycles", LARGEUR_TEXTE+4); //fleches et accents...
732
733 SDL_Window *win = NULL;
734 SDL_Renderer *ren = NULL;
735 if (!init_SDL(&win, &ren)){
736     afficher_ligne("    ERREUR SDL, PAS DE FENETRE", LARGEUR_TEXTE);
737     afficher_bord_bas();
738     return 1;
739 }
740
741 float SCALE = (fmin(LARGEUR_FENETRE, HAUTEUR_FENETRE) * 0.7) / fmax(max_l, max_h);
742 snprintf(buffer, sizeof(buffer), "    SCALE = %f", SCALE);

```

```
743     afficher_ligne(buffer, LARGEUR_TEXTE);
744     afficher_ligne("    Affichage en cours...", LARGEUR_TEXTE);
745     afficher_ligne_vide();
746     main_loop(ren, coeffs, M, SCALE);
747
748     nettoyage_SDL(win, ren);
749
750     free(coeffs);
751     free(valeurs_t);
752
753     afficher_ligne("    Fin affichage.", LARGEUR_TEXTE);
754     afficher_ligne("    Fermeture du programme.", LARGEUR_TEXTE);
755     afficher_bord_bas();
756     return 0;
757 }
```